

一，ES6++

什么是ECMA

// 中文名称是欧洲计算机制造商协会，这个组织的目的是评估，开发和认可电信和计算机标准。1994年后该组织改名为ECMA国际

什么是ECMAScript

// ECMAScript是由ECMA 国际通过ECMA-262标准化的脚本程序设计语言

1，ES6

1.1 let变量声明以及声明特性

```
//变量不能重复声明
let star = 'zll';
let star = 'wxl';//此时会报错

//块级作用域有效 全局、函数、eval
{ //if for else while
  let str='zzl';
}
console.log(str);//此时会报错，因为拿不到

// 不存在变量提升
console.log(name);//会报错，不能使用name这个变量
let name='zzl';

//不影响作用域链
{
  let name='zzl';//虽然是块级作用域，但是不影响作用域链的效果
  function fn(){
    console.log(name);
  }
  fn();//可以拿到
}
```

1.1.1 let const和var的区别？

// var 存在变量提升(在声明语句代码之前可以使用, 不会报错) let const 存在暂时性死区(声明之后在声明语句后面代码才可以用)

```
console.log(a)           console..log(a)  报错           console.log(a) 报错
var a = 10                let a = 10           const a = 10
```

```
// var 声明的变量可以重复声明    let const声明的变量不能重复声明
var a = 10                        let a = 10                        const a = 10
var a = 20                        let a = 20 报错                    const a = 14 报错
// var 没有块级作用域            let const 有块级作用域(例如: 在while for 循环中)
for(var i = 0;i<10;i++){          for(let i = 0;i<10;i++){
i泄漏为全局变量, 存在于全局变量中    i有块级作用域
```

```
// var let 是声明变量, 变量值可以改变    cosnt 声明常量, 值不能改(准确来说, 内存地址不能变),而且声明时必须赋值
```

1.2 const 声明常量以及特点

- 1, const声明常量
- 2, 一定要赋初始值
- 3, 常量的值不能再修改
- 4, 也是块级作用域,
- 5, 声明常量

```
const CL='zzl';
// 对于数组和对象的元素修改, 不算2中的常量修改, 不会报错
const SZ=['zzl','wxl'];
SZ.push('ll');//此时不会报错, 因为只是修改数据, 并没有修改地址
```

1.3 变量的解构赋值

```
//数组解构赋值
const names=['zzl','wxl','ll'];
let [name1,name2,name3]=names;
console.log(name1);//输出zzl

//对象的结构赋值
const data={
  name:'zzl',
  age:'22',
  active:function(){
    console.log('我会飞');
  }
}
let {name,age,active}=data;
console.log(name); // 输出zzl
active(); // 调用方法, 输出我会飞

let {active} = data;
active();
```

1.4 模板字符串

```
let str=`字符串`

// 它可以支持换行
// 变量拼接

let name='zzl';
let data=`${name}是个人` // 'zzl是个人'
```

1.5 对象简化写法

```
// es6允许在大括号里面直接写入变量名和函数名，作为对象的属性和方法

let name='zzl';
let fun=function(){
  console.log('我是人');
}
const data={
  name, //可以直接写变量名，不用再写值了
  fun
}
console.log(data); // 结果输出 name:'zzl',fun()

const data={
  // 旧的方法定义
  // act:function(){
  //   console.log('我会动');
  // }
  //新的方法定义
  act(){
    console.log('我会动');
  }
}
```

1.6 箭头函数以及声明特点

```
// 声明一个函数
let fn=function(a,b){
}
// 箭头函数声明一个函数
let fn=(a,b)=>{
}
fn(1,2);
```

1.6.1 特点

```
// 箭头函数的this是静态的，this始终指向函数声明时所在作用域下的this的值
function getname(){
  //name是zzl,直接调用的话this值指向window
}
console.log(this.name); //zzl
let getname2=()=>{
```

```
//而这个箭头函数是在全局作用域下声明的，所以this也是指向window
console.log(this.name);//zzl
}

window.name='zzl';
const data={
  name:'wxl'
}

//直接调用
getName();//name是zzl,直接调用的话this值指向window
getName2();//而箭头函数是在全局作用域下声明的，所以this也是指向window

// call方法调用
getName.call(data);//此时普通函数this的值已经变为data了。
getName2.call(data);
//输出wxl,因为它的this依然指向函数getName2声明时所在作用域下的this的值window.name='zzl';

////////////////////////////////////

// 不能作为构造实例化对象
let Person=(name,age)=>{
  this.name=name;
  this.age=age;
}
let me=new Person('zzl','22');//此时会报错，箭头函数不能作为实例化对象。

////////////////////////////////////

//不能使用arguments变量
// arguments:用来保存实参的
let fn=()=>{
  console.log(arguments);
}
fn(1,2,3);//会报错，不用用来保存实参
```

1.6.2 简写箭头函数

[illegible]


```

let sum=(n)=>{
    return n*n;
};
console.log(sum(9));

//简写为下面-----

//当代码体只有一条语句的时候，此时return必须省略，而且语句的执行结果就是函数的返回值
let sum=(n)=>n*n;
console.log(sum(9));

//小括号和花括号同时省略-----
let sum=n=>n*n;
console.log(sum(9));

```

1.6.3 箭头函数注意事项

```

// 箭头函数适合与this无关的回调，比如定时器，数组的方法回调。
// 箭头函数不适合与this有关的回调，比如DOM元素的事件回调、对象的方法。

btn.addEventListener("click",function(){
    //此时普通函数的this指向事件源
    //如果使用箭头函数，事件源将变成外部作用域的this值，即这个函数所在的作用域
})

```

1.6.4 箭头函数中的this

```

// 箭头函数不会创建自己的this,它只会从自己的作用域链的上一层继承this

function Person() {
    this.age = 0;

    setInterval(function growUp() {
        this.age++;    // 定时器中使用普通函数，this永远指向window,因为是window调用定时器 所以this.age 值得是Nan
    }, 1000);
}

var p = new Person();
// '-----'
function Person() {
    this.age = 0
    setInterval(()=> {    // 箭头函数没有自己this，只能往上一层去找
        console.log(this) // Person对象
        this.age++;
        console.log(this.age)
    }, 1000);
}
var p = new Person();

```

1.6.5 笔试题

```
var myObject = {
  foo: "bar",
  func: function () {
    var self = this; // self = myObject
    console.log(this.foo); // 'bar'
    console.log(self.foo); // 'bar'
    (function () {
      console.log(this.foo); // undefined // 立即执行函数里面this永远指向window
      console.log(self.foo); // 'bar'
    })();
  }
};
myObject.func();

// -----

function plugin () {
  this.a = 1
  this.b = {
    x: 1,
    confirm: function(title, callback) {
      this.title = title
      console.log(this, '2')
      callback()
    }
  }
  console.log(this, '1')
}
var obj = new plugin()
obj.b.confirm('弹窗', () => {
  console.log(this, '3')
})
```

1.6.6 箭头函数与普通函数的区别？

- 1, **this** 的指向问题
// 箭头函数没有自己的作用域，所以没有自己的**this**，箭头函数中的**this**指向箭头函数当前函数声明时的作用域链的上一层
// 普通函数有自己的作用域
- 2, **call()** 和 **apply()**
// 箭头函数里**this**的指向不受**call**, **apply**的指向，只跟定义位置作用域链的上一层有关，而普通函数可以
- 3, 箭头函数不能当构造函数，而且没有**Prototype**属性，而普通函数可以
- 4, 箭头函数没有**arguments**对象，普通函数有**arguments**对象，箭头函数中的参数可以用...参数名称来获取剩余参数
- 5, 箭头函数不能作为**generator**函数，普通函数可以

1.6.7 箭头函数的this的指向？

```
// 箭头函数里的this指向沿着其作用域链往上找
```

代码的执行结果是什么？

1.7 函数参数初始值

```
function add(a,b,c=10){
}

//函数参数初始值与结构赋值结合使用
function data({name,age='22'}){
  console.log(name);
  console.log(age);
}

data({
  name='zzl',
  // age='23'
})
```

1.7.1 rest参数

```
// 用于获取函数的实参，可以代替arguments。

function data(){
  console.log(arguments); // 输出的是一个对象
}
data('zzl','wxl');

// rest参数
function data(...args){
  console.log(args); //输出的是一个数组，可以使用filter some...
}
data('zzl','wxl');

// -----

// rest参数必须放在参数最后。

function data(a,b,...args){
  console.log(a);//1
  console.log(b);//2
  console.log(args);// [3,4,5]
}
data(1,2,3,4,5)

// -----

const obj1={
```

```

    q: 'zzl'
  }
  const obj2={
    s: 'wxl'
  }
  const data={...obj1,...obj2}; //相当于合并了两个对象
  console.log(data); // {q: 'zzl', s: 'wxl'}

```

1.8 扩展运算符

```

// 它能将数组转换为逗号分隔的参数序列

const names=['zzl','wxl','ll'];
function data(){
  console.log(arguments);
}

//不用扩展运算符
data(names); // 只输出一个结果，是一个数组
// 用扩展运算符
data(...names); // 输出3个结果，等价于：data('zzl','wxl','ll')，即参数序列

// -----
// 如果数组里面有引用类型的话，扩展运算符也只是浅拷贝。
// 可以用来数组的合并
// 数组的克隆
// 伪数组转为真正的数组
// 数组的合并

const a=['zzl'];
const b=['wxl'];
const c=[...a,...b]; // c=['zzl','wxl']

//将伪数组转为真正的数组
const divs=document.querySelectorAll('div'); // 得到伪数组
const divdata=[...divs]; // 转为真正的数组

```

1.8 symbol

```

// 它是一种新的数据类型，表示独一无二的值，类似于字符串的数据类型。

// 它的值是唯一的，用来解决命名冲突的问题。
// 它的值不能于其他数据进行运算。
// 它定义的对象属性不能使用for...in 循环遍历，但是可以使用Reflect.ownKeys来获取对象的所有键名。

// 创建symbol
let s = Symbol();

// 下面的两个symbol对象是不一样的
let s2 = Symbol('aini')
let s3 = Symbol('sini')

// symbol.for创建
let s4 = Symbol.for('aini')
let s5 = Symbol.for('aini')

```

```

let game={
    //假如有很多代码很多变量名
}

//声明一个对象
let data={
    //Symbol保证了up和down的属性名是独一无二的，
    // 所以添加进去也不怕也不怕有属性名冲突

    up:Symbol(),
    //up属性的数据类型为Symbol

    down:Symbol()
};

//第一种添加方式
//把这个Symbol添加到game方法中
game[data.up]=function(){
    console.log('我会飞'); //安全的向这个对象中添加了两个方法
}
game[data.down]=function(){
    console.log('我会爬');
}
console.log(game);

////////////////////////////////////

//第二种添加方式
let play={
    name='run',
    [Symbol('say')]:function(){
        console.log('我会说话');
    },
    [Symbol('sleep')]:function(){
        console.log('我会睡觉');
    }
}
console.log(play);

//补充代码
const title1 = Symbol("title")
const title2 = Symbol("title")
const info={
    [title1]:'zzl',
    [title2]:'wxl'
}
Symbol有很多内置方法

```

1.8.1 Symbol的数据类型及特性？

```
// 数据类型的声明
let sym = Symbol()
// 作用：保证每一个对象的名字都是独一无二的

// 可以接受字符串作为参数，作为对Symbol实例的描述
let sym = Symbol('aini')
// symbol 的参数是一个对象，可以调用该对象的toString 方法将其转换为字符串，然后生成一个symbol值
// symbol函数的参数只是对当前Symbol 值的描述，因此相同参数的Symbol函数返回的值是不相等的
let s1 = Symbol('s1') s2 = Symbol('s1') console.log(s1 === s2) // false

// symbol值不能与其他类型的值进行运算，否则会报错

//symbol 值可以显示转换为字符串和Boolean 但是不能转换为Number

// Object.getOwnPropertySymbols 方法可以获取指定对象的所有Symbol属性名

// symbol声明的属性是没办法通过Object.key() 拿到，所以可以实现私有属性
```

1.8.2 Symbol.iterator的作用？

```
// Symbol.iterator 为每一个对象定义了默认的迭代器。该迭代器可以被for ... of 循环使用
```

1.9 迭代器

任何数据结构只要部署了Iterator接口，就可以使用for...of 来遍历。

1.9.1 具备iterator接口的数据类型

```
// Array
// Arguments
// Set
// Map
// String
// TypedArray
// NodeList

// 这个接口就是对象里面的一个属性，属性的名字叫Symbol.iterator，也可以自己对结构进行布置
// iterator接口。

// -----

for( let i in data){
    i是键名
}
for( let i of data){
    i是键值
}
const arr=['z1','w1'];
console.log(arr);
//arr里面就有Symbol.iterator这个属性。
```

1.9.2 工作原理

```
// 先创建一个指针对象，指向当前数据结构的起始位置
// 第一次调用对象的next方法，指针自动指向数据结构的第一个成员
// 接下来不断调用next方法，指针一直往后移动，直到指向最后一个成员
// 每次调用next方法就会返回一个包含value和done属性（是否完成）的对象

let xiyou = ['唐僧','孙悟空','猪八戒','沙僧']

let iterator = xiyou[Symbol.iterator]();

// 调用对象的next方法
console.log(iterator.next()) // {value:'唐僧',done: false}
console.log(iterator.next()) // {value:'孙悟空',done: false}
console.log(iterator.next()) // {value:'猪八戒',done: false}
console.log(iterator.next()) // {value:'沙僧',done: false}
console.log(iterator.next()) // {value:undefined,done: true}
```

1.9.3 手写迭代器

```
//声明一个对象
const data={
  name:'zzl',
  lis:[
    'wxl',
    'll',
    'hll'
  ],
};

//自己给某些结构加上iterator接口
[Symbol.iterator]() {
  //索引变量
  let index=0;
  let _this=this;
  return {
    //返回一个指针对象，即创建一个指针对象
    next:function(){
      //创建对象的next方法
      // 返回一个包含value和done属性（是否完成）的对象
      if(index < _this.lis.length){
        const result = { value:_this.lis[index], done:false};
        index++;
        return result;
      }else{
        return {value: undefined, done:true};
      }
    }
  };
}

//自定义遍历这个对象
for(let v of data){
  console.log(v);
}

console.log('-----')
```

```
console.log(data);
```

1.9.4 iterator接口的作用?

// **Iterator** 是一个接口，用于处理所有不同的数据结构遍历器。只要数据结构定义了**Iterator**接口时可以用**for of** 来循环遍历

// **for of** 循环的执行过程

```
var arr = ['blue','green','yellow','balck','orange']
let it = iterator(arr)
console.log(it.next())
console.log(it.next())
console.log(it.next())
console.log(it.next())
```

```
function iterator(arr){
  let nextIndex = 0
  return {
    next: function() {
      return nextIndex < arr.length ? {
        value: arr[nextIndex ++] ,done: true
      } : {
        value : undefined,done : false
      }
    }
  }
}
```

// **Iterator** 接口部署在数据结构的**Symbol.iterator**属性

// 一个数据结构只要具有**Symbol.Iterator**属性，就可以认为是'可遍历的'

// **Symbol.Iterator** 属性是一个函数，每次运行会返回一个生成器对象包含俩属性**value**和**done**

// 原生具备**Iterator**接口的数据结构

// **Array** **Map** **Set** **String** 函数的**arguments**对象

// 伪数组转换为真正数组的方法

```
1, Array.prototype.slice.call(arguments)
2, Array.from(arguments)
```

//

1.10 数组的常用方法

1.10.1 forEach()

// 第一个参数：数组元素；

// 第二个参数：数组索引；

// 第三个元素：数组对象本身

```
var numbers = [1,2,3,4,5,4,3,2,1];
numbers.forEach(function(item,index,array){
  console.log(item,index,array[index]);
});
```

// -----

// 为每个元素加 1


```

var data = [1,2,3,4];
data.forEach(function(v,i,a){
    a[i] = v + 1;
    //会改变原数组元素
});
console.log(data);

// 如果只关心数组元素的值，可以只使用一个参数，额外的参数将被忽略

// -----
// 求和
var data = [1,2,3,4];
var sum = 0;
data.forEach(function(value){ //第一个参数数组元素，相当于遍历多有元素；
    sum += value;
    //不会改变原数组元素；
});
console.log(sum);

// -----

// （1）forEach()方法无法在所有元素都被传递给调用的函数之前终止遍历，即没有像 for 循环中使用的 break 语句；
// （2）如果要提前终止，必须把 forEach()方法放在 try 块中，并能抛出一个异常：
function foreach(a,f,t){
    try{
        a.forEach(f, t);
    }
    catch(e){
        if(e === foreach.break) return;
        else throw e;
    }
}
foreach.break = new Error("StopIteration");

```

1.10.2 every() ----- some()

```

// （1）最相似的是 every()和 some()，它们都是数组的逻辑判定：用于判定数组元素是否满足某个条件；
// （2）对 every()，传入的函数必须对每一项都返回 true，这个方法才返回 true；
// （3）而 some()是只要传入的函数对数组中的某一项返回 true，就会返回 true，否则返回 false；

var numbers = [1,2,3,4,5,4,3,2,1];
var everyResult = numbers.every(function(item,index,array){
    return (item > 2);
});
console.log(everyResult); // false

var someResult = numbers.some(function(item,index,array){
    return (item > 2);
});
console.log(someResult); // true

// （4）一旦 every()和 some()确定该返回什么值时，它们就会停止遍历数组元素；
// （5）（some()在判定函数第一次返回 true 后就返回 true，但如果判定函数一直返回 false，它将会遍历整个数组；every()恰好相反，它在判定函数第一次返回 false 后就返回 false，但如果判定函数一直返回 true，它将会遍历整个数组）。

```

```
// (6) 根据数学上的惯例，在空数组上调用时，every()返回 true，some()返回 false;
```

1.10.3 filter()

```
// (1) 返回的是数组元素是调用的数组的一个子集；
// (2) 回调函数是用来逻辑判定的，该函数返回 true 或 false；如果返回的是 true 或真值，则该函数处理的数组元素就被添加到返回的子集数组中；
// (3) filter()方法会跳过稀疏数组中缺少的元素，它的返回数组总是密集的

var numbers = [1,2,3,4,5,4,3,2,1];
var filterResult = numbers.filter(function(item,index,array){
    return item>2;
});
console.log(filterResult);

var evenResult = numbers.filter(function(value,index){
    return index % 2 == 0; // [1, 3, 5, 3, 1] index 是索引
});
console.log(evenResult);

// 压缩稀疏数组
var sparse = [1,,,4];
var dense = sparse.filter(function(){
    return true; //过滤稀疏元素
});
console.log(dense);

// 压缩并删除 undefined 和 null 元素
var sparse = [1,null,3,undefined,,6];
var dense = sparse.filter(function(v){
    return v !== undefined && v !== null;
});
console.log(dense);

// (注：该方法返回的是符合条件的数组元素；)
```

1.10.4 map()

```
// (1) 将调用的数组的每个元素传递给回调函数，并将调用的结果组成一个新数组返回；
// (2) 其不会修改原始数组，如果是稀疏数组，返回的也是相同方式的稀疏数组，即具有相同的长度、相同的缺失元素；

var numbers = [1,2,3,4,5,4,3,2,1];
var mapResult = numbers.map(function(item,index,array){
    return item*2;
});
console.log(mapResult);

//-----
var a = [1,null,3,undefined,5];
var b = a.map(function(v){
    return v * v;
});
console.log(b);
```

```
// -----

//不做任何处理
var spare = [1,,4,null,undefined,NaN,6];
var dense = spare.map(function(v,i,a){
});
console.log(dense);

// -----
// 返回所有数组元素
var spare = [1,,4,null,undefined,NaN,6];
var dense = spare.map(function(v,i,a){
    return v;
});
console.log(dense);

// -----
//能处理的元素进项处理，空元素返回空元素；
var spare = [1,,4,null,undefined,NaN,6];
var dense = spare.map(function(v,i,a){
    return v*v; //进行数据类型转换，由于 null 和 undefined 不能进行数据类型转换});
console.log(dense) //，所以都返回 NaN
```

1.10.5 reduce() ----- reduceRight()

//（1）**reduce()**和 **reduceRight()**；这两个方法都会迭代数组的所有项，然后构建一个最终返回的值；
 //（2）其中，**reduce()**方法从数组的第一项开始，逐个遍历到最后；而 **reduceRight** 则从数组的最后一项开始，向前遍历到第一项；
 //（3）这两个方法都接收两个参数：调用的函数 **callbackfn** 和作为归并基础的初始值**initialValue**（可选的）；
 //（4）这个函数接收 4 个参数：前一个值、当前值、项的索引和数组对象；其返回的任何值都会作为第一个参数自动传给下一项；第一次迭代发生在数组的第二项上，因此第一个参数是数组的第一项，第二个参数就是数组的第二项

```
// 求和
var values = [1,2,3,4];
var sum = values.reduce(function(prev,cur,index,array){
    return prev + cur;
});
console.log(sum); // 10
    // 第一次遍历: prev=1 cur=2;
    // 第二次遍历: prev=3 cur=3;
    // 第三次遍历: prev=6 cur=4;
    // 第四次遍历: prev=10
/*
    说明：第一次执行回调函数，prev 与 cur 分别为数组的第 1 和第 2 项，即 prev 是 1，cur 是 2；第二次，
    prev 是第一次执行的结果，即 3（1+2 的结果），cur 是数组的第三项，即是 3，以此类推；
    reduce()参数函数的参数也可以使用两个参数，如：
*/

//-----
-
// 求和
```

```

var values = [1,2,3,4];
    var sum = values.reduce(function(prev,cur){
return prev + cur;
});
console.log(sum); // 10

// -----
// 求积
var product = values.reduce(function(prev,cur){
    return prev * cur;
});
console.log(product); // 24

//-----
// 求最大值
var max = values.reduce(function(prev,cur){
    return prev > cur ? prev : cur;
});
console.log(max); // 4
// reduce()方法的第二个参数 initialValue 是回调函数的第一个参数 previousValue 的初始值；
如：
// 把以上所有示例添加第二个参数，如：
var values = [1,2,3,4];
var sum = values.reduce(function(prev,cur){
return prev + cur;
},2); // 12

/*
意思就是 pre 不是第一项，而是给 pre 赋一个值，cur 从第一项开始迭代说明，第一次调用时，prev 为
initialValue 值，即为 2，cur 为数组第一项，即为 1；第二次调用，prev 为 3，cur 为 2，以此类
推；reduceRight()的作用类似，只不过方向相反，即按照数组索引从高到低（从右到左）处理数组；如：
*/
var values = [1,2,3,4,5];
var sum = values.reduceRight(function(prev,cur,index,array){
    return prev + cur; //21
},6);
//pre 的初始值为 6；
console.log(sum); // 15

// -----
// 求 2^(3^4)，乘方操作的优先顺序是从右到左
var a = [2,3,4];
var big = a.reduceRight(function(accu, value){
    return Math.pow(value, accu);
});
console.log(big);

//在空数组上，不带初始值参数的 reduce()将导致类型异常；如果数组只有一个元素并且没有指定初始值，
或者一个空数组并且指定了一个初始值，该方法只是简单的返回那个值而不会调用回调函数；

var a = [];
// var b1 = a.reduce(function(x,y){return x + y}); // no initial value 会报错
// var b2 = a.reduce(function(x,y){return x + x}); // 也会报错
var b3 = a.reduce(function(x,y){return x + y},3); //只返回初始值
console.log(b3); // 3

```

// (1) 使用 `reduce()` 还是 `reduceRight()`，主要取决于要从哪头开始遍历数组；除此之外，它们完全相同。

// (2) `reduce()` 和 `reduceRight()` 是典型的函数式编程；有些地方，会把其回调函数称为化简函数，这个化简函数的作用就是用某种方法把两个值组合或化简为一个值，并返回化简后的值；如以上的示例，化简函数通过各种方法，组合了两个值

1.11 生成器

// 生成器就是一个特殊的函数，是异步编程新的解决方案。分别通过以下代码可知运行逻辑。

```
function * fun(){
    console.log('zzl');
}
let a=fun();
// console.log(a); // 输出一个迭代器对象（有next方法）
a.next(); // 输出zzl
```

```
// -----
function * fun(){
    console.log('zzl');
    console.log('wxl');
}
let a=fun();
console.log(a);
// 输出一个迭代器对象（有next方法）
a.next();
// 输出zzl wxl 即全部输出
```

```
// -----
function * fun(){
    console.log('zzl');
    yield '我要暂停';
    console.log('wxl');
}
let a=fun();
console.log(a);
// 输出一个迭代器对象（有next方法）
a.next();
// 输出zzl 我要暂停
```

```
// -----
function * fun(){
    console.log('zzl');
    yield '我要暂停';
    console.log('wxl');
}
let a=fun();
// console.log(a); // 输出一个迭代器对象（有next方法）
a.next(); // 输出zzl
a.next(); // 输出wxl
function * fun(){
    // console.log('zzl');
    yield '我要暂停';
    // console.log('wxl');
}
let a=fun();
console.log(a);
```

```
//输出一个迭代器对象（有next方法）
for(i of fun()){
  console.log(i); //我要暂停
}

// -----
function * fun(){
  // console.log('zzl');
  yield '我要暂停';
  // console.log('wxl');
}

let a=fun();
console.log(a.next()); //输出 {value: '我要暂停', done: false}
```

1.11.1 生成器函数的参数传递

```
function * fun(arg){
  console.log(arg); //输出aaa
  let one = yield 111;
  console.log(one); //输出bbb
  let two = yield 222;
  console.log(two); //输出ccc
  let three = yield 333;
  console.log(three); //输出ddd
}

let a=fun('aaa');
console.log(a.next()); //第一次调用next
//next方法可以传入实参
//第二次调用next的实参将作为第一个yield的整体返回结果
console.log(a.next('bbb')) // 输出{value: 222, done: false}
console.log(a.next('ccc')) // 输出{value: 333, done: false}
console.log(a.next('ddd')) // 输出{value: undefined, done: true}
```

1.11.2 生成器函数实例

```
// 实例1：在控制台每隔一秒分别打印111,222,333
// 回调地狱

setTimeout(() => {
  console.log(111);
  setTimeout(() => {
    console.log(222);
    setTimeout(() => {
      console.log(333);
    }, 3000);
  }, 2000);
}, 1000);

// 解决方法
function one(){
  setTimeout(()=>{
    console.log(111);
    a.next(); //定时器运行完调用下一个，实现了异步编程
  },1000)
}

function two(){
```

```

        setTimeout(()=>{
            console.log(222);
            a.next();
        },2000)
    }
    function three(){
        setTimeout(()=>{
            console.log(333);
            a.next();
        },3000)
    }

    function *fun(){
        yield one();
        yield two();
        yield three();
    }

```

//调用生成器函数

```
let a=fun();
```

```
a.next();
```

结果演示

// -----

// 实例2 模拟获取用户数据，订单数据，商品数据

```

    function one(){
        setTimeout(()=>{
            let data='用户数据';
            a.next(data); //第二次调用next的实参将作为第一个yield的整体返回结果
        },1000)
    }
    function two(){
        setTimeout(()=>{
            let data='订单数据';
            a.next(data);
        },1000)
    }
    function three(){
        setTimeout(()=>{
            let data='商品数据';
            a.next(data);
        },1000)
    }
    function *fun(){
        let users= yield one();
        console.log(users); //用户数据 作为第一个yield的整体返回结果
        let orders= yield two();
        console.log(orders); // 订单数据
        let goods= yield three();
        console.log(goods); // 商品数据
    }
    //调用生成器函数
    let a=fun();
    a.next();

```

1.12 Promise

// 它是es6中异步编程的新的解决方案。相当于一个构造函数。

```
function fun(){
  return new Promise((resolve,reject)=>{
    //如果成功就调用resolve
    let data='数据库中的数据';
    resolve(data);
    //promise状态变为成功

    //如果失败就调用reject
    let err='数据库读取失败';
    reject(err); //promise状态变为失败
  })
}

var promise = fun()
promise.then(
  //调用then方法
  function(success){
    // promise状态变为成功后then会调用第一个回调函数
    console.log(success);
  },function(err){
    // promise状态变为失败后then会调用第二个回调函数
    console.log(err);
  }
)
```

1.12.1 promise封装读取文件

```
const fs = require('fs')

fs.readFile('./aini.txt',(err,data) => {
  // 如果失败，则抛出异常
  if(err) throw err;
  // 如果没哟出错，则输出内容
  console.log(data.toString())
})

// 使用Promise封装
const p = new Promise(function(resolve,reject){
  fs.readFile('./aini.txt',(err,data) => {
    // 如果失败，则抛出异常
    if(err) reject(err)
    // 如果没哟出错，则输出内容
    resolve(data)
  })
})

p.then(function(value){
  console.log(value.toString())
},function(resson){
  console.log('读取失败!!!')
})
```



```
})
```

1.12.2 Promise 封装Ajax请求

```
const p = new Promise(function(resolve, reject){

  const xhr = new XMLHttpRequest()

  xhr.open('GET', 'https://api.apipopen.top/getJoke')

  xhr.send()

  xhr.onreadystatechange = function(){
    if (xhr.readyState == 4){
      // 判断状态码
      if (xhr.status >= 200 && xhr.status < 300){
        // 表示成功
        resolve(xhr.response)
      }else{
        reject(xhr.status)
      }
    }
  }
})

p.then(function(value){
  console.log(value.toString())
}, function(reason){
  console.log(reason)
})
```

1.12.3 Promise.prototype.then()

```
const p = new Promise((resolve, reject){
  setTimeout(() => {
    resolve('用户数据')
  }, 1000)
})

// 调用, then方法的结果也是promise对象, 结果由回调函数的结果决定
// 1, 如果回调函数返回的结果是非promise类型的数据, 状态为成功, 返回值为对象的成功值, 如果不写
// return, 默认返回undefined, 也是成功的promise
const p1 = p.then(value => {
  console.log(value)
  return '11232' // 成功的Promise
}).err => {
  console.warn(reason)
})

// 2, 如果返回的是promise对象, 则这个promise对象的返回结果决定, then方法返回的结果
const p1 = p.then(value => {
  console.log(value)
  return new Promise(function(resolve, reject){
    resolve('ok') // 成功, 则then方法返回结果是成功
  })
})
```

```

        reject('失败了') // 失败，则then方法返回的结果是失败的
    )
  }.err => {
    console.warn(reason)
  })

// 如果抛出错误，then 也是个失败的结果
const p1 = p.then(value => {
  console.log(value)
  throw new Error('出错啦') // then的状态为rejected
}).err => {
  console.warn(reason)
})

// 链式调用
p.then(value=> {
  return new Promise(function(resolve, reject){
    resolve()
  })
}, reason => {

}).then(value=> {
  return new Promise(function(resolve, reject){
    resolve()
  })
}, reason => {

}).then(value=> {
  return new Promise(function(resolve, reject){
    resolve()
  })
}, reason => {

})

```

1.12.4 Promise 多个文件内容读取

```

// 回调地狱
const fs = require('fs')

fs.readFile('./aini.txt', (err, data1) => {
  fs.readFile('./norah.txt', (err, data2) => {
    fs.readFile('./dilnur.txt', (err, data3) => {
      let result = data1 + data2 + data3
    })
  })
})

// 使用promise实现
const p = new Promise(function(resolve, reject){
  fs.readFile('./aini.txt', (err, data) => {
    resolve(data)
  })
})

```

```

p.then(value => {
  return new Promise((resolve, reject) => {
    fs.readFile('./norah.txt', (err, data) => {
      resolve([value, data])
    })
  })
}).then(value => {
  return new Promise((resolve, reject) => {
    fs.readFile('./dilnur.txt', (err, data) => {
      value.push(data)
      resolve(data)
    })
  })
}).then(value => {
  console.log(value.join('\r\n'))
})

```

1.12.5 Promise对象的catch()方法

```

const p = new Promise((resolve, reject) => {
  setTimeout(() => {
    reject('出错了')
  }, 1000)
})

// 第一种
p.then(function(value){
  // 成功的结果
}, function(reason){
  // 失败的结果
})

// 第二种
p.then(function(value){
  // 成功的结果
}).catch(function(reason){
  //失败的结果
})

// 第三种
p.then(function(value){
  // 成功的结果
})

p.catch(function(reason){
  //失败的结果
})

```

1.13 Set

Set的数据类型及特性?

```
// set 是ES6新增的复杂数据类型，类似于数组，但是值没有重复分
// 新建
const s = new Set()
// 可以接受数组或字符串当参数
const s = new Set([1,2,3,4,2,3])
const s1 = new Set('abcdeftg')
// set 可以转化成数组，通过...扩展运算符或者Array.from()方法
let arr = [...s]      // 等同于  Array.from(s)
let arr1 = [...s1]    // 等同于  Array.from(s1)
// set 用于数组去重

// set 里面的判断机制是 === 运算符 (NaN 例外)
let set = new Set()
set.add(5)
set.add('5') // 5 '5' 是两个元素

// set 实例的属性和方法
1, size() 返回长度
2, has() 判断是否包含某个元素
2, forEach 方法
let set = new Set([1,3,2,3,54,3,2])
console.log(set)
set.forEach((item,index) => console.log(item + '-----' + index))
// item 和 key 值一样
// 并集
let arr = [1,2,3,4]
let arr1 = [2,3,4,5,6]
let a = new Set(arr)
let b = new Set(arr1)
let set = new Set([...a,...b])

// 交集
let arr = [1,2,3,4]
let arr1 = [2,3,4,5,6]
let a = new Set(arr)
let b = new Set(arr1)

let intersect = new Set([...a].filter(x => b.has(x)))

// 差集
let arr = [1,2,3,4]
let arr1 = [2,3,4,5,6]
let a = new Set(arr)
let b = new Set(arr1)

let chaji = new Set([...a].filter(x => !b.has(x)))
```

1.14 Map

Map的数据类型及特性?

// Map 是ES6新增的复杂数据类型，类似于对象，也是键值对集合，但是键的范围不限于字符串，各种类型的值都可以当做键

// Map 是值与值的对应，是更完美的键值对结构的提现

// 新建Map数据的方法

```
1, const m = new Map()
```

// Map 的方法

```
1, set() m.set({x: 1}, 22)
```

```
2, get m.get({x: 1})
```

```
3, has() 判断某个键是否存在 m.has({x:1})
```

```
4, delete() 删除某个键值对
```

// 判断是不是Map类型的数据结构

```
let map = new Map()
```

```
1, map instanceof Map // true 通过原型链查找
```

```
2, Object.prototype.toString.call(map) // [object, Map]
```

// 为什么 Object.prototype.toString.call() 用的是原型链顶端的toString方法呢?

// 因为数组和对象对toString方法都进行了改写

// 可以接受数组作为参数

// 注意的是：数组成员是一个个表示键值对的数组

```
let map = new Map(
```

```
[
```

```
  ['name','aini'],
```

```
  ['age',22],
```

```
  [{name:'norah'}, '这是一个对象']
```

```
]
```

```
)
```

// 一个坑

```
map.get({name:'norah'}) // undefined 是因为 {} 与 {} 不是同一个对象
```

// 解决办法：

```
let obj = {name:'norah'}
```

```
let map = new Map(
```

```
[
```

```
  ['name','aini'],
```

```
  ['age',22],
```

```
  [obj, '这是一个对象']
```

```
]
```

```
)
```

```
map.get(obj) // 这样才能获取正确的key值
```

// map参数的扩展

// 任何具有 Iterator 接口，且每个成员都是一个双元素的数组的数据结构，都可以当做map函数的参数

1.15 Class类

// 以手机类为例

```
function Phone(brand,price){
```

```
  this.brand = brand
```

```
  this.price = price
```

```

}
// 静态成员
//相当于 python里的staticmethod
Phone.name = '手机'
Phone.change = function(){
    console.log('我可以改变世界')
}

// 相当于Python里的classmethod
Phone.prototype.size = "5.5英寸"
Phone.prototype.call = function() {
    console.log('我可以打电话')
}

let Huawei = new Phone('华为',5999)

// ES6语法
class Phone{
    // 构造函数方法
    constructor(brand,price){
        this.brand = brand
        this.call = call
    }

    // 语法必须使用这个方法，不能用ES5老语法
    call() {
        console.log('我可以打电话')
    }
}

let Huawei = new Phone('华为',5999)

```

1.15.1 构造函数继承

```

function Phone(brand,price){
    this.brand = brand
    this.price = price
}

Phone.prototype.call = function(){
    console.log('我可以打电话')
}

//智能手机
function SmartPhone(brand,price,color,size){
    Phone.call(this,brand,price){
        this.color = color
        this.size = size
    }
}

// 设置子级构造函数的原型
SmartPhone.prototype = new Phone
SmartPhone.prototype.constructor = SmartPhone

// 声明子类的方法
SmartPhone.prototype.photo = function(){

```

```

    console.log('我可以拍照')
  }

  SmartPhone.prototype.playGame = function() {
    console.log('我可以打游戏')
  }

  const chuzi = new SmartPhone("锤子",2499,'黑色','505')

```

1.15.2 ES6里类的继承

```

class Phone{
  constructor(brand,price){
    this.brand = brand
    this.price = price
  }

  call(){
    console.log('我可以打电话')
  }
}

class SmartPhone extends Phone{
  constructor(brand,price,color,size){
    super(brand,price)
    this.color = color
    this.size = size
  }
  photo() {
    console.log('我可以拍照')
  }
  playGame() {
    console.log('我可以打游戏')
  }
}

const xiaomi = new SmartPhone('xiaomi',2499,'黑色','505')

```

1.15.3 子类对父类方法的重写

```

class Phone{
  constructor(brand,price){
    this.brand = brand
    this.price = price
  }

  call(){
    console.log('我可以打电话')
  }
}

class SmartPhone extends Phone{
  constructor(brand,price,color,size){
    super(brand,price)
    this.color = color
    this.size = size
  }
}

```

```

    }
    photo() {
        console.log('我可以拍照')
    }
    playGame() {
        console.log('我可以打游戏')
    }
}

// 可以对父类方法重写
call(){
    console.log('我可以进行视频通话')
}
}

const xiaomi = new SmartPhone('xiaomi', 2499, '黑色', '505')

```

1.15.4 class中的getter和setter

```

class Phone{
    get price() {
        console.log('我被读取了')
        return 'i love you '
    }

    set price(val){
        console.log('价格修改了')
        this.val = val
    }
}

let s = new Phone()
console.log(s.price)
// '我被读取了'
// 'i love you ' -----> 这才是s.price的返回值

s.price = 'free' // '价格修改了'

```

1.16 数值扩展

```

// Number.EPSILON ----- 是JavaScript表示的最下精度
// Number.isFinite ----- 检测一个数是否为有限数
// Number.isNaN ----- 检测一个数值是否为 NaN
// Number.parseInt Number.parseFloat----- 字符串转整数
// Number.isInteger ----- 判断一个数是否为整数
// Math.trunc ----- 将数字的小数部分抹掉
// Math.sign ----- 判断一个数到底为正数，负数还是0 -----
-----> 1 -1 0

// 进制数的表示方法
let b = 0b1010
let o = 0o777
let d = 100
let x = 0xffff

```


1.17 对象方法扩展

```
// Object.is -----> 判断两个值是否完全相等
    console.log(Object.is(NaN,NaN)) // true
    console.log(NaN === NaN) // false
// Object.assign -----> 对象的合并
// Object.setPrototypeOf -----> 设置Object原型对象
// Object.getPrototypeOf-----> 获取对象的属性
```

1.18 模块化

// 模块化是指将一个大的文件，拆分成许多小的文件，然后将小文件组合起来，一个个小文件就是一个模块。

```
// 防止命名冲突
// 代码复用
// 高维护性
```

1.18.1 ES Module

```
//第一种暴露方法：分别暴露
export let a='zzl'; //向外暴露
export function fun(){
    console.log('我是谁');//向外暴露
}

//第二种暴露方法：统一暴露[ 注意{}导出的并不是对象 ]
let a='zzl';
export{a,fun};

//第三种：默认暴露
export default{
    a:'zzl',
    fun:function(){
        console.log('我是谁');
    }
}
```

1.18.2 html代码引入js模块

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <script type="module">
        //第一种引入方式：引入mi.js暴露的模块
        import * as m from './模块化/m1.js';
        console.log(m.a); //zzl
```

```

//第二种引入方式解构赋值形式：引入mi.js暴露的模块
import {a as a1, fun} from './模块化/m1.js';
console.log(a1); //可以给a一个别名a1 输出zzl
//默认暴露的引入比较特殊，引入语法是

import * as m from './模块化/m1.js';
m.default.fun();
//默认暴露的解构赋值

import {default as mm} from './模块化/m1.js';
console.log(mm);
//简便形式 针对默认暴露

import m from './模块化/m1.js';
//动态调用模块，即使用时再导入

import('./模块化/m1.js').then(module =>{
    module.fun();
})
</script>
</body>
</html>

```

1.18.3 module.exports 与 exports的使用与区别

```

// 二者都是专门负责导出的东西,都属于commonJS规范,是同步加载的.

// exports
a.js
const a=10

//导出 (实际上exports就是一个对象)
exports._a = a

b.js

// 引入a.js导出的对象,实际就是exports对象
const ba =require('./a.js') // === exports对象
console.log('输出',ba._a) //输出 10

// exports
a.js
const a=10
//导出 (实际上exports就是一个对象)
// exports._a=a
/**
 * 实际上module也是一个对象
 * 下边三行代码具体逻辑如下:
 * 1、module.exports = exports 即4行的exports,
 *    此时是引用赋值,并且可以继续使用原来b.js的导入方式
 * 2、重新给module.exports一个新对象地址{_a:a}
 * 3、b.js导入方式不变,因为只是对象地址,该对象module.exports依然是导出对象
 */
module.exports = {
    _a:a
}
b.js

```

```
// 引入a.js导出的对象,实际就是module.exports对象
const ba=require('./a.js') // === module.exports对象

console.log('输出',ba._a) //输出 10
```

1.18.4 require()与import()

```
if (true){
  import * from './a.js'
}

/*
以上代码是错误的,因为imprt关键字不支持在这类代码里,但是require支持,因为require本质是函数,但是,require不支持在浏览器里运行,并且它是同步执行的,所以ESModule里可以用import函数来引入外部模块,并且它是异步执行的.
*/

if(true){
  import('./a.js').then(res=>{
    console.log(res._a)
  }).catch(err=>{
    console.log(err)
  })
}
// <script src type="module"> 也是异步加载执行的,相当于加了async属性
```

1.19 Babel

```
// 它可将js的新语法转化为旧语法,以便更好的兼容。

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  // 安装工具 babel-cli babel-preset-env browserify(打包工具)
  // 初始化 npm init -yes
  // 安装 npm i babel-cli babel-preset-env browserify -D
  // 因为局部安装,用 npx 执行命令
  // npx babel aini.js(源文件目录) -d dist/js(打包的目标目录) --
presers=babel-preset-env
  // 打包 npx browserify dist/js/app.js -o dist/bundle.js
</body>
</html>
```

2, ES7

2.1 新特性

```
Array.prototype.includes // 检测数组中是否包含某个元素
const name = ['aini','norah','dilnur']
console.log(name.includes('aini')) // true
** // 幂次方
console.log(2 ** 10) // 1024    Math.pow(2,10) 一样的效果
```

3, ES8

3.1 async函数

```
// async 函数的返回值为promise对象
// promise对象的结果由 async函数执行的返回值决定

async function fn() {
  return "aini"

  // 如果跑错误，则返回的是失败的promise对象
  // 如果返回的是一个promise对象，结果取决于返回的promise对象的状态
}

const result = fn();
console.log(result) // promise对象
```

3-2 await 表达式

```
// await 必须写在async函数中
// await 右侧的表达式一般为promise对象
// await 返回的是promise成功的值
// await 的promise失败了，就会抛出异常，需要通过try...catch捕获处理

const p = new Promise((resolve,reject) => {
  resolve('用户数据')
})

async function fn() {
  const result = await p
  console.log(result) // '用户数据'
}

fn()

// -----
-----

async function fn() {
  try{
    const result = await p
    console.log(result) // '用户数据'
  }catch(e){
    console.log(e)
  }
}
```

```
    }  
    fn()
```

3-3 async和await结合

```
const fs = require("fs");  
  
function readWeixue() {  
    return new Promise((resolve, reject) => {  
        fs.readFile('/为学.md', (err, data) => {  
            if (err) {  
                reject(err)  
            } else {  
                resolve(data)  
            }  
        })  
    })  
}  
  
function readChayangshi() {  
    return new Promise((resolve, reject) => {  
        fs.readFile('/插秧诗.md', (err, data) => {  
            if (err) {  
                reject(err)  
            } else {  
                resolve(data)  
            }  
        })  
    })  
}  
  
function readGuanShuYouGan() {  
    return new Promise((resolve, reject) => {  
        fs.readFile('/观书有感.md', (err, data) => {  
            if (err) {  
                reject(err)  
            } else {  
                resolve(data)  
            }  
        })  
    })  
}  
  
async function main() {  
    let weixue = await readWeixue()  
    let chayang = await readChayangshi()  
    let guanshu = await readGuanShuYouGan()  
    console.log(weixue.toString())  
    console.log(chayang.toString())  
    console.log(guanxue.toString())  
}
```

3-4 对象方法的扩展

```
Object.values() // 返回一个给定对象所有可枚举属性值的数组
Object.entries() // 返回一个给定对象自身可遍历属性的数组
Object.getOwnPropertyDescriptors() // 返回指定对象自身属性的描述对象
```

4, ES9

4.1 扩展运算符和rest参数

```
function connect({host, port, ...user}){
  console.log(host) // 127.0.0.1
  console.log(port) // 3306
  console.log(user) // {username: 'aini', password: 'aini'}
}

connect({
  host: '127.0.0.1',
  port: 3306,
  username: 'aini',
  password: 'aini'
})

const a = {
  q: 'aini'
}
const b = {
  w: 'norah'
}
const c = {
  e: 'dilnur'
}
const d = {
  r: 'zhuozi'
}

// 对象的合并
const name = {...a, ...b, ...c, ...d}
console.log(name) // {q: 'aini', w: 'norah', e: 'dilnur', r: 'zhuozi'}
```

二, JQuery

1, jquery 入门

1.1 jquery 的优点

```
/*
轻量级。核心文件才几十kb，不会影响页面加载速度
跨浏览器兼容。基本兼容了现在主流的浏览器
链式编程、隐式迭代
对事件、样式、动画支持，大大简化了DOM操作
支持插件扩展开发。有着丰富的第三方的插件，例如：
树形菜单、日期控件、轮播图等
免费、开源
*/
```

1.2 jQuery 的基本使用

1.2.1 jQuery 的入口函数

```
$(function () {
    ... // 此处是页面 DOM 加载完成的入口
});

$(document).ready(function(){
    ... // 此处是页面DOM加载完成的入口
})
```

1. 等着 DOM 结构渲染完毕即可执行内部代码，不必等到所有外部资源加载完成，jQuery 帮我们完成了封装。
2. 相当于原生 js 中的 DOMContentLoaded。
3. 不同于原生 js 中的 load 事件是等页面文档、外部的 js 文件、css文件、图片加载完毕才执行内部代码。
4. 更推荐使用第一种方式。

1.3 jQuery 的顶级对象 \$

```
/*
1.$ 是 jQuery 的别称，在代码中可以使用 jQuery 代替 $，但一般为了方便，通常都直接使用 $。

2.$ 是jQuery 的顶级对象， 相当于原生JavaScript中的 window。把元素利用$包装成jQuery对象，就可以调用
jQuery 的方法。
*/
```

1.4 jQuery 对象和 DOM 对象

```
/*
1. 用原生 JS 获取来的对象就是 DOM 对象
2. jQuery 方法获取的元素就是 jQuery 对象。
3. jQuery 对象本质是： 利用$对DOM 对象包装后产生的对象（伪数组形式存储）。

注意：
只有 jQuery 对象才能使用 jQuery 方法，DOM 对象则使用原生的 JavaScript 方法。
*/

// DOM 对象与 jQuery 对象之间是可以相互转换的。
// 因为原生js 比 jQuery 更大，原生的一些属性和方法 jQuery没有给我们封装。要想使用这些属性和方法需要把jQuery对象转换为DOM对象才能使用
```

1. DOM 对象转换为 jQuery 对象: `$(DOM对象)`
`$('div')`
2. jQuery 对象转换为 DOM 对象 (两种方式)
`$('div')[index]` `index` 是索引号
`$('div').get(index)` `index` 是索引号

2, jQuery 常用API

2.1, jQuery 选择器

2.1.1 jQuery 基础选择器

名称	用法	描述
ID选择器	<code>\$("#id")</code>	获取指定ID的元素
全选选择器	<code>\$("*")</code>	匹配所有元素
类选择器	<code>\$(".class")</code>	获取同一类class的元素
标签选择器	<code>\$("div")</code>	获取同一类标签的所有元素
并集选择器	<code>\$("div,p,li")</code>	选取多个元素
交集选择器	<code>\$("li.current")</code>	交集元素

2.1.2 jQuery 层级选择器

名称	用法	描述
子代选择器	<code>\$("ul>li");</code>	使用>号, 获取亲儿子层级的元素; 注意, 并不会获取孙子层级的元素
后代选择器	<code>\$("ul li");</code>	使用空格, 代表后代选择器, 获取ul下的所有li元素, 包括孙子等

2.1.3 jQuery 筛选选择器

语法	用法	描述
<code>:first</code>	<code>\$('li:first')</code>	获取第一个li元素
<code>:last</code>	<code>\$('li:last')</code>	获取最后一个li元素
<code>:eq(index)</code>	<code>\$("li:eq(2)")</code>	获取到的li元素中, 选择索引号为2的元素, 索引号index从0开始。
<code>:odd</code>	<code>\$("li:odd")</code>	获取到的li元素中, 选择索引号为奇数的元素
<code>:even</code>	<code>\$("li:even")</code>	获取到的li元素中, 选择索引号为偶数的元素

2.1.4 jQuery 筛选方法 (重点)

语法	用法	说明
<code>parent()</code>	<code>\$("#li").parent();</code>	查找父级
<code>children(selector)</code>	<code>\$("#ul").children("li")</code>	相当于 <code>\$("#ul>li")</code> ，最近一级（亲儿子）
<code>find(selector)</code>	<code>\$("#ul").find("li");</code>	相当于 <code>\$("#ul li")</code> ，后代选择器
<code>siblings(selector)</code>	<code>\$(".first").siblings("li");</code>	查找兄弟节点，不包括自己本身
<code>nextAll([expr])</code>	<code>\$(".first").nextAll();</code>	查找当前元素之后所有的同辈元素
<code>prevAll([expr])</code>	<code>\$(".last").prevAll();</code>	查找当前元素之前所有的同辈元素
<code>hasClass(class)</code>	<code>\$('#div').hasClass("protected")</code>	检查当前的元素是否含有某个特定的类，如果有，则返回true
<code>eq(index)</code>	<code>\$("#li").eq(2);</code>	相当于 <code>\$("#li:eq(2)")</code> ，index 从0开始

2.1.5 Query 里面的排他思想

// 想要多选一的效果，排他思想：当前元素设置样式，其余的兄弟元素清除样式。

```
$(this).css('color','red');
$(this).siblings().css('color','');
```

2.1.6 链式编程

```
$(this).css('color','red').siblings().css('color','');
```

2.2, jQuery 样式操作

2.2.1 操作 css 方法

//jQuery 可以使用 `css` 方法来修改简单元素样式； 也可以操作类，修改多个样式。

1. 参数只写属性名，则是返回属性值
`$(this).css('color');`
2. 参数是属性名，属性值，逗号分隔，是设置一组样式，属性必须加引号，值如果是数字可以不用跟单位和引号
`$(this).css('color','red');`
3. 参数可以是对象形式，方便设置多组样式。属性名和属性值用冒号隔开， 属性可以不用加引号，
`$(this).css({ "color":"white","font-size":"20px"});`

2.2.2 设置类样式方法

// 作用等同于以前的 `classList`，可以操作类样式， 注意操作类里面的参数不要加点。

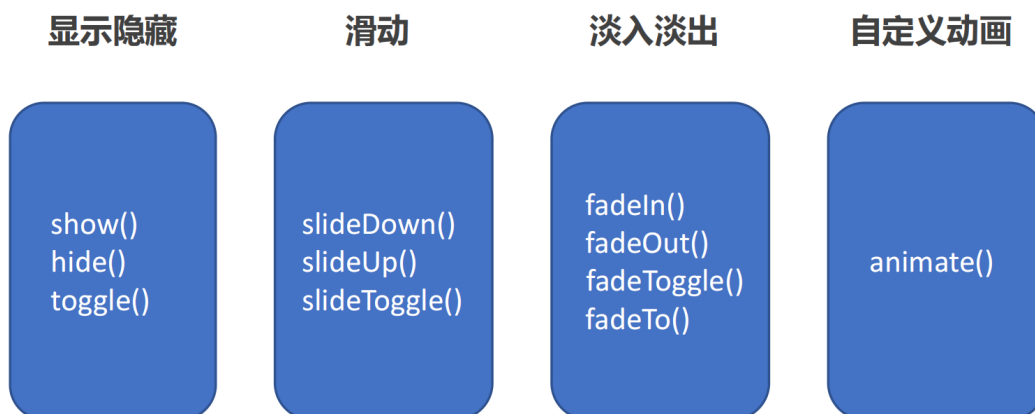
1. 添加类
`$('#div').addClass('current')`
2. 移除类
`$('#div').removeClass('current')`
3. 切换类
`$('#div').toggleClass('current')`

2.3 类操作与className区别

```
// 原生 JS 中 className 会覆盖元素原先里面的类名。  
// jQuery 里面类操作只是对指定类进行操作，不影响原先的类名。
```

2.3, jQuery 效果

jQuery 给我们封装了很多动画效果，最为常见的如下：



2.3.1 显示隐藏效果

建议：平时一般不带参数，直接显示隐藏即可。

1-1 显示语法规范

```
/*  
1. 显示语法规范  
   show([speed],[easing],[fn])  
  
2. 显示参数  
3.1 显示隐藏效果  
   (1) 参数都可以省略， 无动画直接显示。  
   (2) speed: 三种预定速度之一的字符串(“slow”, “normal”, or “fast”)或表示动画时长的毫秒数值(如: 1000)。  
   (3) easing: (Optional) 用来指定切换效果，默认是“swing”，可用参数“linear”。  
   (4) fn: 回调函数，在动画完成时执行的函数，每个元素执行一次。  
*/
```

1-2 隐藏语法规范

```
/*
1. 隐藏语法规范
    hide([speed],[easing],[fn]))

2. 隐藏参数
    (1) 参数都可以省略， 无动画直接显示。
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的
        毫秒数值(如: 1000)。
    (3) easing: (Optional) 用来指定切换效果，默认是“swing”，可用参数“linear”。
    (4) fn: 回调函数，在动画完成时执行的函数，每个元素执行一次。

*/
```

1-3 切换语法规范

```
/*
1. 切换语法规范
    toggle([speed],[easing],[fn]))

2. 切换参数
    (1) 参数都可以省略， 无动画直接显示。
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的
        毫秒数值(如: 1000)。
    (3) easing: (Optional) 用来指定切换效果，默认是“swing”，可用参数“linear”。
    (4) fn: 回调函数，在动画完成时执行的函数，每个元素执行一次。

*/
```

2.3.2 滑动效果

2-1 下滑效果语法规范

```
/*
1. 下滑效果语法规范
    slideDown([speed],[easing],[fn]))

2. 下滑效果参数
    (1) 参数都可以省略。
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的
        毫秒数值(如: 1000)。
    (3) easing: (Optional) 用来指定切换效果，默认是“swing”，可用参数“linear”。
    (4) fn: 回调函数，在动画完成时执行的函数，每个元素执行一次。

*/
```

2-2 上滑效果语法规范

```
/*
1. 上滑效果语法规范
    slideUp([speed],[easing],[fn]))

2. 上滑效果参数
    (1) 参数都可以省略。
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的
        毫秒数值(如: 1000)。
    (3) easing: (Optional) 用来指定切换效果，默认是“swing”，可用参数“linear”。
    (4) fn: 回调函数，在动画完成时执行的函数，每个元素执行一次。

*/
```

2-3 滑动切换效果语法规范

```
/*
1. 滑动切换效果语法规范
    slideToggle([speed],[easing],[fn]])

2. 滑动切换效果参数
    (1) 参数都可以省略。
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的毫秒数值(如: 1000)。
    (3) easing: (Optional) 用来指定切换效果, 默认是“swing”, 可用参数“linear”。
    (4) fn: 回调函数, 在动画完成时执行的函数, 每个元素执行一次。

*/
```

2.3.3 事件切换

```
/*\
    hover([over],[out])

    (1) over: 鼠标移到元素上要触发的函数(相当于mouseenter)
    (2) out: 鼠标移出元素要触发的函数(相当于mouseleave)
    (3) 如果只写一个函数, 则鼠标经过和离开都会触发它

*/
```

2.3.4 动画队列及其停止排队方法

4-1 动画或效果队列

```
// 动画或者效果一旦触发就会执行, 如果多次触发, 就造成多个动画或者效果排队执行
```

4-2 停止排队

```
stop()

// (1) stop() 方法用于停止动画或效果。
// (2) 注意: stop() 写到动画或者效果的前面, 相当于停止结束上一次的动画。
```

2.3.5 淡入淡出效果-1

5-1 淡入效果语法规范

```
/*
1. 淡入效果语法规范
    fadeIn([speed],[easing],[fn]])

2. 淡入效果参数
    (1) 参数都可以省略。
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的毫秒数值(如: 1000)。
    (3) easing: (Optional) 用来指定切换效果, 默认是“swing”, 可用参数“linear”。
    (4) fn: 回调函数, 在动画完成时执行的函数, 每个元素执行一次。

*/
```

5-2 淡出效果语法规范

```
/*  
1. 淡出效果语法规范  
    fadeOut([speed],[easing],[fn]])  
  
2. 淡出效果参数  
    (1) 参数都可以省略。  
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的  
        毫秒数值(如: 1000)。  
    (3) easing: (Optional) 用来指定切换效果, 默认是“swing”, 可用参数“linear”。  
    (4) fn: 回调函数, 在动画完成时执行的函数, 每个元素执行一次。  
*/
```

5-3 淡入淡出切换效果语法规范

```
/*  
1. 淡入淡出切换效果语法规范  
    fadeToggle([speed],[easing],[fn]])  
  
2. 淡入淡出切换效果参数  
    (1) 参数都可以省略。  
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的  
        毫秒数值(如: 1000)。  
    (3) easing: (Optional) 用来指定切换效果, 默认是“swing”, 可用参数“linear”。  
    (4) fn: 回调函数, 在动画完成时执行的函数, 每个元素执行一次。  
*/
```

2.3.6 淡入淡出效果-2

渐进方式调整到指定的不透明度

```
/*  
1. 渐进方式调整到指定的不透明度  
    fadeTo([[speed],opacity,[easing],[fn]])  
  
2. 效果参数  
    (1) opacity 透明度必须写, 取值 0~1 之间。  
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的  
        毫秒数值(如: 1000)。必须写  
    (3) easing: (Optional) 用来指定切换效果, 默认是“swing”, 可用参数“linear”。  
    (4) fn: 回调函数, 在动画完成时执行的函数, 每个元素执行一次。。  
*/
```

3.3.7 自定义动画 animate

```
/*
1. 语法
    animate(params,[speed],[easing],[fn])
2. 参数
    (1) params: 想要更改的样式属性，以对象形式传递，必须写。 属性名可以不用带引号， 如果是复合属性则需要采用驼峰命名法 borderLeft。其余参数都可以省略。
    (2) speed: 三种预定速度之一的字符串(“slow”,“normal”, or “fast”)或表示动画时长的毫秒数值(如: 1000)。
    (3) easing: (Optional) 用来指定切换效果，默认是“swing”，可用参数“linear”。
    (4) fn: 回调函数，在动画完成时执行的函数，每个元素执行一次。
*/
```

2.4, jQuery 属性操作

2.4.1 设置或获取元素固有属性值 prop()

```
1. 获取属性语法
    prop('属性')

2. 设置属性语法
    prop('属性', '属性值')
```

2.4.2 设置或获取元素自定义属性值 attr()

// 用户自己给元素添加的属性，我们称为自定义属性。 比如给 `div` 添加 `index =“1”`。

```
1. 获取属性语法
    attr('属性') // 类似原生 getAttribute()

2. 设置属性语法
    attr('属性', '属性值') // 类似原生 setAttribute()
```

2.4.3 数据缓存 data()

// `data()` 方法可以在指定的元素上存取数据，并不会修改 DOM 元素结构。一旦页面刷新，之前存放的数据都将被移除。

```
1. 附加数据语法
    data('name','value') // 向被选元素附加数据

2. 获取数据语法
    date('name') // 向被选元素获取数据
```

// 同时，还可以读取 HTML5 自定义属性 `data-index`，得到的是数字型

2.5, jQuery 文本属性值

2.5.1 普通元素内容 html() (相当于原生inner HTML)

```
html() // 获取元素的内容  
  
html('内容') // 设置元素的内容
```

2.5.2 普通元素文本内容 text() (相当与原生 innerText)

```
text() // 获取元素的文本内容  
  
text('文本内容') // 设置元素的文本内容
```

2.5.3 表单的值 val() (相当于原生value)

```
val() // 获取表单的值  
  
val('内容') // 设置表单的值
```

2.6, jQuery 元素操作

2.6.1 遍历元素

1-1 语法1

```
$("#div").each(function (index, domEle) { xxx; })
```

1. each() 方法遍历匹配的每一个元素。主要用DOM处理。 each 每一个
2. 里面的回调函数有2个参数: index 是每个元素的索引号; domEle 是每个DOM元素对象, 不是jquery对象
3. 所以要想使用jquery方法, 需要给这个dom元素转换为jquery对象 \$(domEle)

1-2 语法2

```
$.each(object, function (index, element) { xxx; })
```

1. \$.each()方法可用于遍历任何对象。主要用于数据处理, 比如数组, 对象
2. 里面的函数有2个参数: index 是每个元素的索引号; element 遍历内容

2.6.2 创建元素

```
$('<li></li>');
```

2.6.3 添加元素

3-1 内部添加

```
element.append('内容')  
// 把内容放入匹配元素内部最后面，类似原生 appendChild。  
  
element.prepend('内容')  
// 把内容放入匹配元素内部最前面
```

3-2 外部添加

```
element.after('内容') // 把内容放入目标元素后面  
element.before('内容') // 把内容放入目标元素前面  
  
// ① 内部添加元素，生成之后，它们是父子关系。  
// ② 外部添加元素，生成之后，他们是兄弟关系。
```

2.6.4 删除元素

```
element.remove() // 删除匹配的元素（本身）  
element.empty() // 删除匹配的元素集合中所有的子节点  
element.html('') // 清空匹配的元素内容  
  
// ① remove 删除元素本身。  
// ② empty() 和 html('') 作用等价，都可以删除元素里面的内容，只不过 html 还可以设置内容。
```

2.7, jQuery 尺寸、位置操作

2.7.1 Query 尺寸

语法	用法
width() / height()	取得匹配元素宽度和高度值 只算 width / height
innerWidth() / innerHeight()	取得匹配元素宽度和高度值 包含 padding
outerWidth() / outerHeight()	取得匹配元素宽度和高度值 包含 padding、border
outerWidth(true) / outerHeight(true)	取得匹配元素宽度和高度值 包含 padding、border、margin

```
// 以上参数为空，则是获取相应值，返回的是数字型。  
// 如果参数为数字，则是修改相应值。  
// 参数可以不必写单位
```

2.7.2 jQuery 位置

```
// 位置主要有三个： offset()、position()、scrollTop()/scrollLeft()
```


2-1 offset() 设置或获取元素偏移

```
// offset() 方法设置或返回被选元素相对于文档的偏移坐标，跟父级没有关系。  
// 该方法有2个属性 left、top。offset().top 用于获取距离文档顶部的距离，offset().left 用于获取距离文档左侧的距离。  
// 可以设置元素的偏移：offset({ top: 10, left: 30 });
```

2-2 position() 获取元素偏移

```
// position() 方法用于返回被选元素相对于带有定位的父级偏移坐标，如果父级都没有定位，则以文档为准。  
// 该方法有2个属性 left、top。position().top 用于获取距离定位父级顶部的距离，position().left 用于获取距离定位父级左侧的距离。  
// 该方法只能获取。
```

2-3 scrollTop()/scrollLeft() 设置或获取元素被卷去的头部和左侧

```
// scrollTop() 方法设置或返回被选元素被卷去的头部。  
// 不跟参数是获取，参数为不带单位的数字则是设置被卷去的头部。
```

3, jQuery 事件

3.1, jQuery 事件注册

3.1.1 单个事件注册

```
element.事件(function(){})  
$("div").click(function(){ 事件处理程序 })  
  
//其他事件和原生基本一致。  
//比如mouseover、mouseout、blur、focus、change、keydown、keyup、resize、scroll 等
```

3.2, jQuery 事件处理

3.2.1 事件处理 on() 绑定事件

```
// on() 方法在匹配元素上绑定一个或多个事件的事件处理函数  
  
element.on(events,[selector],fn)  
  
1. events: 一个或多个用空格分隔的事件类型，如"click"或"keydown"。  
2. selector: 元素的子元素选择器。  
3. fn: 回调函数 即绑定在元素身上的侦听函数。
```

1-1 on() 方法优势1:

```
// 可以绑定多个事件，多个处理事件处理程序。
$('div').on({
    mouseover: function() {},
    mouseout: function() {},
    click: function() {}
});

// 如果事件处理程序相同
$('div').on("mouseover mouseout", function() {
    $(this).toggleClass("current");
});
```

1-2 on() 方法优势2:

// 可以事件委派操作。事件委派的定义就是，把原来加给子元素身上的事件绑定在父元素身上，就是把事件委派给父元素。

```
$( 'ul' ).on( 'click', 'li', function() {
    alert( 'hello world!' );
});
```

//在此之前有bind(), live() delegate()等方法来处理事件绑定或者事件委派，最新版本的请用on替代他们。

1-3 on() 方法优势3:

```
// 动态创建的元素，click() 没有办法绑定事件， on() 可以给动态生成的元素绑定事件
$('div').on("click", 'p', function() {
    alert("俺可以给动态生成的元素绑定事件")
});

$("div").append($("<p>我是动态创建的p</p>"));
```

3.2.2 事件处理 off() 解绑事件

// off() 方法可以移除通过 on() 方法添加的事件处理程序。

```
$( "p" ).off() // 解绑p元素所有事件处理程序
$( "p" ).off( "click" ) // 解绑p元素上面的点击事件 后面的 foo 是侦听函数名
$( "ul" ).off( "click", "li" ); // 解绑事件委托
```

// 如果有的事件只想触发一次， 可以使用 one() 来绑定事件。

3.3.3 自动触发事件 trigger()

// 有些事件希望自动触发，比如轮播图自动播放功能跟点击右侧按钮一致。可以利用定时器自动触发右侧按钮点击事件，不必鼠标点击触发。

```
element.click() // 第一种简写形式

element.trigger("type") // 第二种自动触发模式

$( "p" ).on( "click", function () {
```

```
    alert("hi~");  
});
```

```
$("#p").trigger("click"); // 此时自动触发点击事件，不需要鼠标点击
```

// 有些事件希望自动触发，比如轮播图自动播放功能跟点击右侧按钮一致。可以利用定时器自动触发右侧按钮点击事件，不必鼠标点击触发。

```
element.triggerHandler(type) // 第三种自动触发模式
```

```
// triggerHandler模式不会触发元素的默认行为，这是和前面两种的区别
```

3.3, jQuery 事件对象

// 事件被触发，就会有事件对象的产生。

```
element.on(events,[selector],function(event) {  
    // 阻止默认行为：  
    event.preventDefault()  
    // 或者  
    return false  
  
    // 阻止冒泡：  
    event.stopPropagation()  
  
})
```

4, jQuery 其他方法

4.1, jQuery 拷贝对象

//如果想要把某个对象拷贝（合并） 给另外一个对象使用，此时可以使用 `$.extend()` 方法

```
$.extend([deep], target, object1, [objectN])
```

1. **deep**: 如果设为`true` 为深拷贝， 默认为`false` 浅拷贝
2. **target**: 要拷贝的目标对象
3. **object1**:待拷贝到第一个对象的对象。
4. **objectN**:待拷贝到第N个对象的对象。
5. 浅拷贝是把被拷贝的对象复杂数据类型中的地址拷贝给目标对象，修改目标对象会影响被拷贝对象。
6. 深拷贝，前面加`true`， 完全克隆(拷贝的对象,而不是地址)，修改目标对象不会影响被拷贝对象。

4.2, 多库共存

问题概述:

// jQuery使用\$作为标识符, 随着jQuery的流行, 其他 js 库也会用这\$作为标识符, 这样一起使用会引起冲突。

客观需求:

// 需要一个解决方案, 让jQuery 和其他的js库不存在冲突, 可以同时存在, 这就叫做多库共存。

jQuery 解决方案:

1. 把里面的 \$ 符号 统一改为 jQuery。比如 `jQuery('div')`
 2. jQuery 变量规定新的名称: `$.noConflict()`
- ```
var xx = $.noConflict();
```

## 三, Ajax

### 1, 服务器的基本概念与初识Ajax

#### 1.1 URL地址的组成部分

URL地址一般由三部组成:

- ① 客户端与服务器之间的通信协议
- ② 存有该资源的服务器名称
- ③ 资源在服务器上具体的存放位置

`http://www.cnblogs.com/liulongbinblogs/p/11649393.html`

通信协议      服务器名称      资源在服务器上具体的存放位置

#### 1.2 客户端与服务器的通信过程

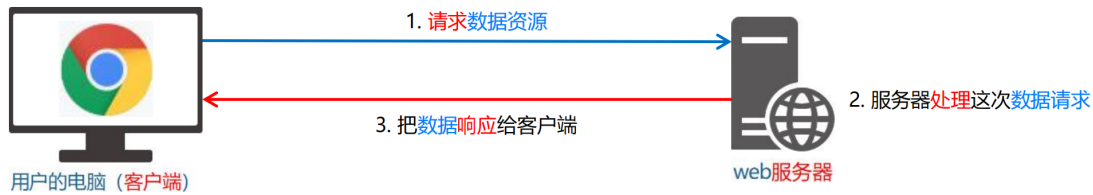


注意:

- ① 客户端与服务器之间的通信过程, 分为 请求 - 处理 - 响应 三个步骤。
- ② 网页中的每一个资源, 都是通过 请求 - 处理 - 响应 的方式从服务器获取回来的。

## 1.3 网页中如何请求数据

数据，也是服务器对外提供的一种资源。只要是资源，必然要通过 请求 - 处理 - 响应 的方式进行获取。



如果要在网页中请求服务器上的数据资源，则需要用到 XMLHttpRequest 对象。

XMLHttpRequest (简称 xhr) 是浏览器提供的 js 成员，通过它，可以请求服务器上的数据资源。

最简单的用法 `var xhrObj = new XMLHttpRequest()`

## 1.4 \$.get()函数

### 1.4.1 \$.get()函数的语法

// jQuery 中 \$.get() 函数的功能单一，专门用来发起 get 请求，从而将服务器上的资源请求到客户端来进行使用。\$.get() 函数的语法如下：

`$.get(url, [data], [callback])`

| 参数名      | 参数类型     | 是否必选 | 说明           |
|----------|----------|------|--------------|
| url      | string   | 是    | 要请求的资源地址     |
| data     | object   | 否    | 请求资源期间要携带的参数 |
| callback | function | 否    | 请求成功时的回调函数   |

### 1.4.2 \$.get()发起不带参数的请求

// 使用 \$.get() 函数发起不带参数的请求时，直接提供请求的 URL 地址和请求成功之后的回调函数即可，示例代码如下：

```
$.get('http://www.liulongbin.top:3006/api/getbooks', function(res) {
 console.log(res) // 这里的 res 是服务器返回的数据
})
```

### 1.14.3 \$.get()发起带参数的请求

```
$.get('http://www.liulongbin.top:3006/api/getbooks', { id: 1 }, function(res) {
 console.log(res)
})
```

## 1.5 \$.post()函数

### 1.5.1 \$.post()函数的语法

```
// jQuery 中 $.post() 函数的功能单一，专门用来发起 post 请求，从而向服务器提交数据。
$.post() 函数的语法如下：
$.post(url, [data], [callback])
```

其中，三个参数各自代表的含义如下：

| 参数名      | 参数类型     | 是否必选 | 说明           |
|----------|----------|------|--------------|
| url      | string   | 是    | 提交数据的地址      |
| data     | object   | 否    | 要提交的数据       |
| callback | function | 否    | 数据提交成功时的回调函数 |

### 1.5.2 \$.post()向服务器提交数据

```
// 使用 $.post() 向服务器提交数据的示例代码如下：
$.post(
 'http://www.liulongbin.top:3006/api/addbook', // 请求的URL地址
 { bookname: '水浒传', author: '施耐庵', publisher: '上海图书出版社' }, // 提交的数
 据
 function(res) { // 回调函数
 console.log(res)
 }
)
```

## 1.6 \$.ajax()函数的语法

```
// 相比于 $.get() 和 $.post() 函数，jQuery 中提供的 $.ajax() 函数，是一个功能比较综合的函
数，它允许我们对Ajax 请求进行更详细的配置。
// $.ajax() 函数的基本语法如下：
$.ajax({
 type: '', // 请求的方式，例如 GET 或 POST
 url: '', // 请求的 URL 地址
 data: { }, // 这次请求要携带的数据
 success: function(res) { } // 请求成功之后的回调函数
})
```

### 1.6.1 使用\$.ajax()发起GET请求

```
// 使用 $.ajax() 发起 GET 请求时，只需要将 type 属性的值设置为 'GET' 即可：
$.ajax({
 type: 'GET', // 请求的方式
 url: 'http://www.liulongbin.top:3006/api/getbooks', // 请求的 URL 地址
 data: { id: 1 }, // 这次请求要携带的数据
 success: function(res) { // 请求成功之后的回调函数
 console.log(res)
 }
})
```

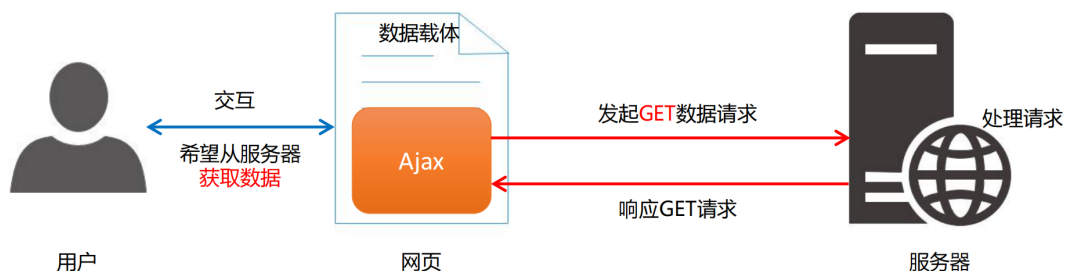
## 1.6.2 使用\$.ajax()发起POST请求

```
// 使用 $.ajax() 发起 POST 请求时，只需要将 type 属性的值设置为 'POST' 即可：
$.ajax({
 type: 'POST', // 请求的方式
 url: 'http://www.liulongbin.top:3006/api/addbook', // 请求的 URL 地址
 data: { // 要提交给服务器的数据
 bookname: '水浒传',
 author: '施耐庵',
 publisher: '上海图书出版社'
 },
 success: function(res) { // 请求成功之后的回调函数
 console.log(res)
 }
})
```

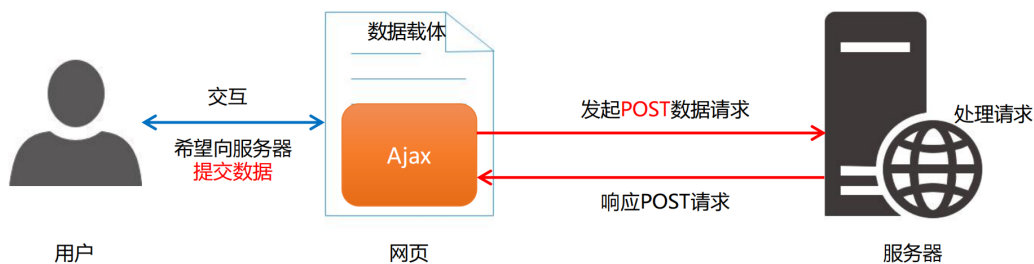
## 1.7 接口

### 1.7.1 通过GET方式请求接口的过程

#### 1. 通过GET方式请求接口的过程



### 1.7.2 通过POST方式请求接口的过程



## 2, Ajax加强

### 2.1 使用xhr发起GET请求

```
// 步骤：
① 创建 xhr 对象
② 调用 xhr.open() 函数
③ 调用 xhr.send() 函数
④ 监听 xhr.onreadystatechange 事件
```

```
// 1. 创建 XHR 对象
var xhr = new XMLHttpRequest()

// 2. 调用 open 函数，指定 请求方式 与 URL地址
xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks')

// 3. 调用 send 函数，发起 Ajax 请求
xhr.send()

// 4. 监听 onreadystatechange 事件
xhr.onreadystatechange = function() {
 // 4.1 监听 xhr 对象的请求状态 readyState；与服务器响应的状态 status
 if (xhr.readyState === 4 && xhr.status === 200) {
 // 4.2 打印服务器响应回来的数据
 console.log(xhr.responseText)
 }
}
```

### 2.1.1 了解xhr对象的readyState属性

XMLHttpRequest 对象的 readyState 属性，用来表示**当前 Ajax 请求所处的状态**。每个 Ajax 请求必然处于以下状态中的一个：

| 值 | 状态               | 描述                                    |
|---|------------------|---------------------------------------|
| 0 | UNSENT           | XMLHttpRequest 对象已被创建，但尚未调用 open方法。   |
| 1 | OPENED           | open() 方法已经被调用。                       |
| 2 | HEADERS_RECEIVED | send() 方法已经被调用，响应头也已经被接收。             |
| 3 | LOADING          | 数据接收中，此时 response 属性中已经包含部分数据。        |
| 4 | DONE             | Ajax 请求完成，这意味着数据传输已经彻底 <b>完成或失败</b> 。 |

### 2.1.2 使用xhr发起带参数的GET请求

使用 xhr 对象发起带参数的 GET 请求时，只需在调用 xhr.open 期间，为 URL 地址指定参数即可：

```
// ...省略不必要的代码
xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks?id=1')
// ...省略不必要的代码
```

这种在 URL 地址后面拼接的参数，叫做查询字符串

## 2.2 使用xhr发起POST请求

步骤：

- ① 创建 xhr 对象
- ② 调用 xhr.open() 函数
- ③ 设置 Content-Type 属性（固定写法）
- ④ 调用 xhr.send() 函数，同时指定要发送的数据
- ⑤ 监听 xhr.onreadystatechange 事件

```
// 1. 创建 xhr 对象
var xhr = new XMLHttpRequest()
// 2. 调用 open()
xhr.open('POST', 'http://www.liulongbin.top:3006/api/addbook')
// 3. 设置 Content-Type 属性（固定写法）
xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded')
// 4. 调用 send(), 同时将数据以查询字符串的形式，提交给服务器
```



```
xhr.send('bookname=水浒传&author=施耐庵&publisher=天津图书出版社')
// 5. 监听 onreadystatechange 事件
xhr.onreadystatechange = function() {
 if (xhr.readyState === 4 && xhr.status === 200) {
 console.log(xhr.responseText)
 }
}
```

## 2.3 json

// 对象结构：对象结构在 JSON 中表示为 { } 括起来的内容。数据结构为 { key: value, key: value, ... } 的键值对结构。其中，key 必须是使用英文的双引号包裹的字符串，value 的数据类型可以是

// 数字、字符串、布尔值、null、数组、对象6种类型。

### 2.3.1 JSON语法注意事项

// ① 属性名必须使用双引号包裹  
 // ② 字符串类型的值必须使用双引号包裹  
 // ③ JSON 中不允许使用单引号表示字符串  
 // ④ JSON 中不能写注释  
 // ⑤ JSON 的最外层必须是对象或数组格式  
 // ⑥ 不能使用 undefined 或函数作为 JSON 的值  
 // JSON 的作用：在计算机与网络之间存储和传输数据。  
 // JSON 的本质：用字符串来表示 Javascript 对象数据或数组数据

### 2.3.2 序列化和反序列化

// 把数据对象转换为字符串的过程，叫做序列化，例如：调用 JSON.stringify() 函数的操作，叫做 JSON 序列化。  
 // 把字符串转换为数据对象的过程，叫做反序列化，例如：调用 JSON.parse() 函数的操作，叫做 JSON 反序列化。

## 2.4 封装自己的Ajax函数

// 定义options参数选项  
 // itheima() 函数是我们自定义的 Ajax 函数，它接收一个配置对象作为参数，配置对象中可以配置如下属性：

- 1, method 请求的类型
- 2, url 请求的 URL 地址
- 3, data 请求携带的数据
- 4, success 请求成功之后的回调函数

// 处理data参数，得到查询字符串  
 // 需要把 data 对象，转化成查询字符串的格式，从而提交给服务器，因此提前定义 resolveData 函数如下：

```
/**
 * 处理 data 参数
 * @param {data} 需要发送到服务器的数据
 * @returns {string} 返回拼接好的查询字符串 name=zs&age=10
 */
function resolveData(data) {
 var arr = []
 for (var k in data) {
```

```

arr.push(k + '=' + data[k])
}
return arr.join('&') // 获得查询字符串
}

// 判断请求的类型
// 不同的请求类型，对应 xhr 对象的不同操作，因此需要对请求类型进行 if ... else ... 的判断
if (options.method.toUpperCase() === 'GET') {
 // 发起 GET 请求
 xhr.open(options.method, options.url + '?' + qs)
 xhr.send()
} else if (options.method.toUpperCase() === 'POST') {
 // 发起 POST 请求
 xhr.open(options.method, options.url)
 xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded')
 xhr.send(qs)
}

// 定义aini函数
function aini(options) {
 var xhr = new XMLHttpRequest()
 // 拼接查询字符串
 var qs = resolveData(options.data)
 // 判断请求的类型
 if (options.method.toUpperCase() === 'GET') {
 // 发起 GET 请求
 xhr.open(options.method, options.url + '?' + qs)
 xhr.send()
 } else if (options.method.toUpperCase() === 'POST') {
 // 发起 POST 请求
 xhr.open(options.method, options.url)
 xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded')
 xhr.send(qs)
 }
 // 监听请求状态改变的事件
 xhr.onreadystatechange = function() {
 if (xhr.readyState === 4 && xhr.status === 200) {
 var result = JSON.parse(xhr.responseText)
 options.success(result)
 }
 }
}
}

```

## 2.5 XMLHttpRequest Level2的新特性

### 2.5.1 设置HTTP请求时限

// 有时, Ajax 操作很耗时, 而且无法预知要花多少时间。如果网速很慢, 用户可能要等很久。新版本的 XMLHttpRequest 对象, 增加了 timeout 属性, 可以设置 HTTP 请求的时限:

```
xhr.timeout = 3000
```

// 上面的语句, 将最长等待时间设为 3000 毫秒。过了这个时限, 就自动停止HTTP请求。与之配套的还有一个timeout 事件, 用来指定回调函数:

```
xhr.ontimeout = function(event){
alert('请求超时!')
}
```

## 2.5.2 FormData对象管理表单数据

1, Ajax 操作往往用来提交表单数据。为了方便表单处理, HTML5 新增了一个 FormData 对象, 可以模拟表单操作:

// 1. 新建 FormData 对象

```
var fd = new FormData()
```

// 2. 为 FormData 添加表单项

```
fd.append('uname', 'zs')
```

```
fd.append('upwd', '123456')
```

// 3. 创建 XHR 对象

```
var xhr = new XMLHttpRequest()
```

// 4. 指定请求类型与URL地址

```
xhr.open('POST', 'http://www.liulongbin.top:3006/api/formdata')
```

// 5. 直接提交 FormData 对象, 这与提交网页表单的效果, 完全一样

```
xhr.send(fd)
```

2, FormData对象也可以用来获取网页表单的值, 示例代码如下:

// 获取表单元素

```
var form = document.querySelector('#form1')
```

// 监听表单元素的 submit 事件

```
form.addEventListener('submit', function(e) {
```

```
 e.preventDefault()
```

```
 // 根据 form 表单创建 FormData 对象, 会自动将表单数据填充到 FormData 对象中
```

```
 var fd = new FormData(form)
```

```
 var xhr = new XMLHttpRequest()
```

```
 xhr.open('POST', 'http://www.liulongbin.top:3006/api/formdata')
```

```
 xhr.send(fd)
```

```
 xhr.onreadystatechange = function() {}
```

```
})
```

## 2.6 上传文件

// 新版 XMLHttpRequest 对象, 不仅可以发送文本信息, 还可以上传文件。

实现步骤:

① 定义 UI 结构

② 验证是否选择了文件

③ 向 FormData 中追加文件

- ④ 使用 `xhr` 发起上传文件的请求
- ⑤ 监听 `onreadystatechange` 事件

### 1. 定义UI结构

```
<!-- 1. 文件选择框 -->
<input type="file" id="file1" />
<!-- 2. 上传按钮 -->
<button id="btnUpload">上传文件</button>

<!-- 3. 显示上传到服务器上的图片 -->

```

### 2. 验证是否选择了文件

```
// 1. 获取上传文件的按钮
var btnUpload = document.querySelector('#btnUpload')
// 2. 为按钮添加 click 事件监听
btnUpload.addEventListener('click', function() {
// 3. 获取到选择的文件列表
var files = document.querySelector('#file1').files
if (files.length <= 0) {
return alert('请选择要上传的文件!')
}
// ...后续业务逻辑
})
```

### 3. 向FormData中追加文件

```
// 1. 创建 FormData 对象
var fd = new FormData()
// 2. 向 FormData 中追加文件
fd.append('avatar', files[0])
```

### 4. 使用 xhr 发起上传文件的请求

```
// 1. 创建 xhr 对象
var xhr = new XMLHttpRequest()
// 2. 调用 open 函数，指定请求类型与URL地址。其中，请求类型必须为 POST
xhr.open('POST', 'http://www.liulongbin.top:3006/api/upload/avatar')
// 3. 发起请求
xhr.send(fd)
```

### 5. 监听onreadystatechange事件

```
xhr.onreadystatechange = function() {
 if (xhr.readyState === 4 && xhr.status === 200) {
 var data = JSON.parse(xhr.responseText)
 if (data.status === 200) { // 上传文件成功
 // 将服务器返回的图片地址，设置为 标签的 src 属性
 document.querySelector('#img').src =
 'http://www.liulongbin.top:3006' + data.url
 } else { // 上传文件失败
 console.log(data.message)
 }
 }
}
```

## 2.7 显示文件上传进度

// 新版本的 XMLHttpRequest 对象中, 可以通过监听 xhr.upload.onprogress 事件, 来获取到文件的上传进度。

语法格式如下:

// 创建 XHR 对象

```
var xhr = new XMLHttpRequest()
```

// 监听 xhr.upload 的 onprogress 事件

```
xhr.upload.onprogress = function(e) {
```

// e.lengthComputable 是一个布尔值, 表示当前上传的资源是否具有可计算的长度

```
if (e.lengthComputable) {
```

// e.loaded 已传输的字节

// e.total 需传输的总字节

```
var percentComplete = Math.ceil((e.loaded / e.total) * 100)
```

```
}
```

```
}
```

### 1. 导入需要的库

```
<link rel="stylesheet" href="./lib/bootstrap.css" />
```

```
<script src="./lib/jquery.js"></script>
```

### 2. 基于Bootstrap渲染进度条

<!-- 进度条 -->

```
<div class="progress" style="width: 500px; margin: 10px 0;">
```

```
 <div class="progress-bar progress-bar-info progress-bar-striped active"
id="percent" style="width: 0%">
```

```
 0%
```

```
 </div>
```

```
</div>
```

### 3. 监听上传进度的事件

```
xhr.upload.onprogress = function(e) {
```

```
 if (e.lengthComputable) {
```

// 1. 计算出当前上传进度的百分比

```
var percentComplete = Math.ceil((e.loaded / e.total) * 100)
```

```
$('#percent')
```

// 2. 设置进度条的宽度

```
.attr('style', 'width:' + percentComplete + '%')
```

// 3. 显示当前的上传进度百分比

```
.html(percentComplete + '%')
```

```
}
```

```
}
```

### 4. 监听上传完成的事件

```
xhr.upload.onload = function() {
```

```
 $('#percent')
```

// 移除上传中的类样式

```
.removeClass()
```

// 添加上传完成的类样式

```
.addClass('progress-bar progress-bar-success')
```

```
}
```

## 2.8 jQuery实现文件上传

### 1. 定义UI结构

```
<!-- 导入 jQuery -->
<script src="./lib/jquery.js"></script>
<!-- 文件选择框 -->
<input type="file" id="file1" />
<!-- 上传文件按钮 -->
<button id="btnUpload">上传</button>
```

### 2. 验证是否选择了文件

```
$('#btnUpload').on('click', function() {
 // 1. 将 jQuery 对象转化为 DOM 对象，并获取选中的文件列表
 var files = $('#file1')[0].files
 // 2. 判断是否选择了文件
 if (files.length <= 0) {
 return alert('请选择图片后再上传！')
 }
})
```

### 3. 向FormData中追加文件

```
// 向 FormData 中追加文件
var fd = new FormData()
fd.append('avatar', files[0])
```

### 4. 使用jQuery发起上传文件的请求

```
$.ajax({
 method: 'POST',
 url: 'http://www.liulongbin.top:3006/api/upload/avatar',
 data: fd,
 // 不修改 Content-Type 属性，使用 FormData 默认的 Content-Type 值
 contentType: false,
 // 不对 FormData 中的数据进行 url 编码，而是将 FormData 数据原样发送到服务器
 processData: false,
 success: function(res) {
 console.log(res)
 }
})
```

## 2.9 jQuery实现loading效果

### 1. ajaxStart(callback)

//Ajax 请求开始时，执行 ajaxStart 函数。可以在 ajaxStart 的 callback 中显示 loading 效果，示例代码如下：

```
// 自 jQuery 版本 1.8 起，该方法只能被附加到文档
$(document).ajaxStart(function() {
 $('#loading').show()
})
```

注意： \$(document).ajaxStart() 函数会监听当前文档内所有的 Ajax 请求。

### 2. ajaxStop(callback)

// Ajax 请求结束时，执行 ajaxStop 函数。可以在 ajaxStop 的 callback 中隐藏 loading 效果，示例代码如下：

```
// 自 jQuery 版本 1.8 起，该方法只能被附加到文档
```

```
$(document).ajaxStop(function() {
 $('#loading').hide()
})
```

## 2.10 Axios

### 2.10.1 axios发起GET请求

```
// axios 发起 get 请求的语法:
axios.get('url', { params: { /*参数*/ } }).then(callback)

//具体的请求示例如下:
// 请求的 URL 地址
var url = 'http://www.liulongbin.top:3006/api/get'
// 请求的参数对象
var paramsObj = { name: 'zs', age: 20 }
// 调用 axios.get() 发起 GET 请求
axios.get(url, { params: paramsObj }).then(function(res) {
 // res.data 是服务器返回的数据
 var result = res.data
 console.log(res)
})
```

### 2.10.2 直接使用axios发起POST请求

```
// axios 发起 post 请求的语法:
axios.post('url', { /*参数*/ }).then(callback)

具体的请求示例如下:
// 请求的 URL 地址
var url = 'http://www.liulongbin.top:3006/api/post'
// 要提交到服务器的数据
var dataObj = { location: '北京', address: '顺义' }
// 调用 axios.post() 发起 POST 请求
axios.post(url, dataObj).then(function(res) {
 // res.data 是服务器返回的数据
 var result = res.data
 console.log(result)
})
```

### 2.10.3 直接使用axios发起请求

```
// axios 也提供了类似于 jquery 中 $.ajax() 的函数，语法如下:
axios({
 method: '请求类型',
 url: '请求的URL地址',
 data: { /* POST数据 */ },
 params: { /* GET参数 */ }
}).then(callback)
```

1. 直接使用axios发起GET请求

```
axios({
 method: 'GET',
 url: 'http://www.liulongbin.top:3006/api/get',
 params: { // GET 参数要通过 params 属性提供
 name: 'zs',
 age: 20
 }
}).then(function(res) {
 console.log(res.data)
})
```

2. 直接使用axios发起POST请求

```
axios({
 method: 'POST',
 url: 'http://www.liulongbin.top:3006/api/post',
 data: { // POST 数据要通过 data 属性提供
 bookname: '程序员的自我修养',
 price: 666
 }
}).then(function(res) {
 console.log(res.data)
})
```

## 3 表单的基本使用

### 3.1

#### 标签的属性

属性	值	描述
action	URL地址	规定当提交表单时，向何处发送表单数据
method	get或post	规定以何种方式把表单数据提交到 action URL
enctype	application/x-www-form-urlencoded multipart/form-data text/plain	规定在发送表单数据之前如何对其进行编码
target	_blank _self _parent _top framename	规定在何处打开 action URL

#### 3.1.1 target

值	描述
_blank	在新窗口中打开。
_self	默认。在相同的框架中打开。
_parent	在父框架集中打开。（很少用）
_top	在整个窗口中打开。（很少用）
framename	在指定的框架中打开。（很少用）



### 3.1.2 enctype

```
// 在涉及到文件上传的操作时，必须将 enctype 的值设置为 multipart/form-data
// 如果表单的提交不涉及到文件上传操作，则直接将 enctype 的值设置为 application/x-www-form-urlencoded 即可！
```

值	描述
application/x-www-form-urlencoded	在发送前编码所有字符（默认）
multipart/form-data	不对字符编码。 在使用包含文件上传控件的表单时，必须使用该值。
text/plain	空格转换为 “+” 加号，但不对特殊字符编码。（很少用）

## 3.2 表单的同步提交及缺点

1. 什么是表单的同步提交  
// 通过点击 **submit** 按钮，触发表单提交的操作，从而使页面跳转到 **action URL** 的行为，叫做表单的同步提交。
2. 表单同步提交的缺点  
// 页面会发生跳转  
// 页面之前的状态和数据会丢失
3. 解决方案：表单只负责采集数据，**Ajax** 负责将数据提交到服务器。

## 3.3 通过Ajax提交表单数据

### 3.3.1 监听表单提交事件

// 在 **jQuery** 中，可以使用如下两种方式，监听到表单的提交事件：

```
$('#form1').submit(function(e) {
 alert('监听到了表单的提交事件')
})

$('#form1').on('submit', function(e) {
 alert('监听到了表单的提交事件')
})
```

### 3.3.2 阻止表单默认提交行为

// 当监听到表单的提交事件以后，可以调用事件对象的 **event.preventDefault()** 函数，来阻止表单的提交和页面的跳转，示例代码如下：

```
$('#form1').submit(function(e) {
 // 阻止表单的提交和页面的跳转
 e.preventDefault()
})

$('#form1').on('submit', function(e) {
 // 阻止表单的提交和页面的跳转
 e.preventDefault()
})
```

### 3.3.3 快速获取表单中的数据

#### serialize()函数

```
// 为了简化表单中数据的获取操作，jQuery 提供了 serialize() 函数，其语法格式如下]

$(selector).serialize()

// serialize() 函数的好处：可以一次性获取到表单中的所有数据。
// 注意：在使用 serialize() 函数快速获取表单数据时，必须为每个表单元素添加 name 属性！

<form id="form1">
<input type="text" name="username" />
<input type="password" name="password" />
<button type="submit">提交</button>
</form>

$('#form1').serialize()
// 调用的结果：
// username=用户名的值&password=密码的值
```

## 4 跨域与JSONP

### 4.1 同源策略

#### 4.1.1 什么是同源

```
// 如果两个页面的协议，域名和端口都相同，则两个页面具有相同的源
```

#### 4.1.2 什么是同源策略

```
// 同源策略（英文全称 Same origin policy）是浏览器提供的一个安全功能。
/*
MDN 官方给定的概念：同源策略限制了从同一个源加载的文档或脚本如何与来自另一个源的资源进行交互。这是一个用于隔离潜在恶意文件的重要安全机制。
*/

// 通俗的理解：浏览器规定，A 网站的 JavaScript，不允许和非同源的网站 C 之间，进行资源的交互，
例如：

无法读取非同源网页的 Cookie、LocalStorage 和 IndexedDB
无法接触非同源网页的 DOM
无法向非同源地址发送 Ajax 请求
```

### 4.2 跨域

#### 4.2.1 什么是跨域

```
// 同源指的是两个 URL 的协议、域名、端口一致，反之，则是跨域。
// 出现跨域的根本原因：浏览器的同源策略不允许非同源的 URL 之间进行资源的交互。
```

## 4.2.2 浏览器对跨域请求的拦截



注意：浏览器允许发起跨域请求，但是，跨域请求回来的数据，会被浏览器拦截，无法被页面获取到！

## 4.3 jsonp

### 4.3.1 什么是JSONP

// JSONP (JSON with Padding) 是 JSON 的一种“使用模式”，可用于解决主流浏览器的跨域数据访问的问题。

### 4.3.2 JSONP的实现原理

```
/*
由于浏览器同源策略的限制，网页中无法通过 Ajax 请求非同源的接口数据。但是 <script> 标签不受浏览器同
源策略的影响，可以通过 src 属性，请求非同源的 js 脚本。

因此，JSONP 的实现原理，就是通过 <script> 标签的 src 属性，请求跨域的数据接口，并通过函数调用
的形式，
接收跨域接口响应回来的数据。
*/
```

### 4.3.3 JSONP的缺点

// 由于 JSONP 是通过 <script> 标签的 src 属性，来实现跨域数据获取的，所以，JSONP 只支持 GET 数据请求，不支持 POST 请求。

// 注意：JSONP 和 Ajax 之间没有任何关系，不能把 JSONP 请求数据的方式叫做 Ajax，因为 JSONP 没有用到 XMLHttpRequest 这个对象。

### 4.3.4 jQuery中的JSONP

// jQuery 提供的 \$.ajax() 函数，除了可以发起真正的 Ajax 数据请求之外，还能够发起 JSONP 数据请求，例如：

```
$.ajax({
 url: 'http://ajax.frontend.itheima.net:3006/api/jsonp?name=zs&age=20',
 // 如果要使用 $.ajax() 发起 JSONP 请求，必须指定 dataType 为 jsonp
 dataType: 'jsonp',
 success: function(res) {
 console.log(res)
 }
})
```

```
// 默认情况下，使用 jQuery 发起 JSONP 请求，会自动携带一个 callback=jQueryxxx 的参数，jQueryxxx 是随机生成的一个回调函数名称。
```

```
// -----

```

```
// 自定义参数及回调函数名称
$.ajax({
 url: 'http://ajax.frontend.itheima.net:3006/api/jsonp?name=zs&age=20',
 dataType: 'jsonp',
 // 发送到服务端的参数名称，默认值为 callback
 jsonp: 'callback',
 // 自定义的回调函数名称，默认值为 jQueryxxx 格式
 jsonpCallback: 'abc',
 success: function(res) {
 console.log(res)
 }
})
```

### 4.3.5 jQuery中JSONP的实现过程

```
//jQuery 中的 JSONP，也是通过 <script> 标签的 src 属性实现跨域数据访问的，只不过，jQuery 采用的是动态创建和移除 <script> 标签的方式，来发起 JSONP 数据请求。
```

```
// 在发起 JSONP 请求的时候，动态向 <header> 中 append 一个 <script> 标签；
// 在 JSONP 请求成功以后，动态从 <header> 中移除刚才 append 进去的 <script> 标签；
```

## 4.4 防抖和节流

### 4.4.1 什么是节流

```
// 节流策略（throttle），顾名思义，可以减少一段时间内事件的触发频率。
```

### 4.4.2 节流的应用场景

```
// 鼠标连续不断地触发某事件（如点击），只在单位时间内只触发一次；
// 懒加载时要监听计算滚动条的位置，但不必每次滑动都触发，可以降低计算的频率，而不必去浪费 CPU 资源；
```

### 4.4.3 节流阀的概念

```
/*
```

```
节流阀为空，表示可以执行下次操作；不为空，表示不能执行下次操作。
当前操作执行完，必须将节流阀重置为空，表示可以执行下次操作了。
每次执行操作前，必须先判断节流阀是否为空。
```

```
*/
```

### 4.4.4 使用节流优化鼠标跟随效果

```
$(function() {
 var angel = $('#angel')
 var timer = null // 1.预定义一个 timer 节流阀
 $(document).on('mousemove', function(e) {
 if (timer) { return } // 3.判断节流阀是否为空，如果不为空，则证明距离上次执行间隔不足16毫秒
 timer = setTimeout(function() {
 $(angel).css('left', e.pageX + 'px').css('top', e.pageY + 'px')
 timer = null // 2.当设置了鼠标跟随效果后，清空 timer 节流阀，方便下次开启延时器
 }, 16)
 })
})
```

#### 4.4.5 总结防抖和节流的区别

```
// 防抖：如果事件被频繁触发，防抖能保证只有最有一次触发生效！前面 N 多次的触发都会被忽略！
// 节流：如果事件被频繁触发，节流能够减少事件触发的频率，因此，节流是有选择性地执行一部分事件！
```

## 5,HTTP协议加强

### 5.1 HTTP协议简介

#### 5.1.1 什么是通信

```
// 通信，就是信息的传递和交换。
```

```
// 通信三要素：
```

- 1 通信的主体
- 1 通信的内容
- 1 通信的方式

#### 5.1.2 互联网中的通信

```
//案例：服务器把传智专修学院的简介通过响应的方式发送给客户端浏览器。其中，
/*
通信的主体是服务器和客户端浏览器；
通信的内容是传智专修学院的简介；
通信的方式是响应；
*/
```

#### 5.1.3 什么是通信协议

```
// 通信协议（Communication Protocol）是指通信的双方完成通信所必须遵守的规则和约定。
// 通俗的理解：通信双方采用约定好的格式来发送和接收消息，这种事先约定好的通信格式，就叫做通信协议。
```

#### 5.1.4 互联网中的通信协议

```
//客户端与服务器之间要实现网页内容的传输，则通信的双方必须遵守网页内容的传输协议。
```

```
//网页内容又叫做超文本，因此网页内容的传输协议又叫做超文本传输协议（HyperText Transfer Protocol），简称 HTTP 协议。
```

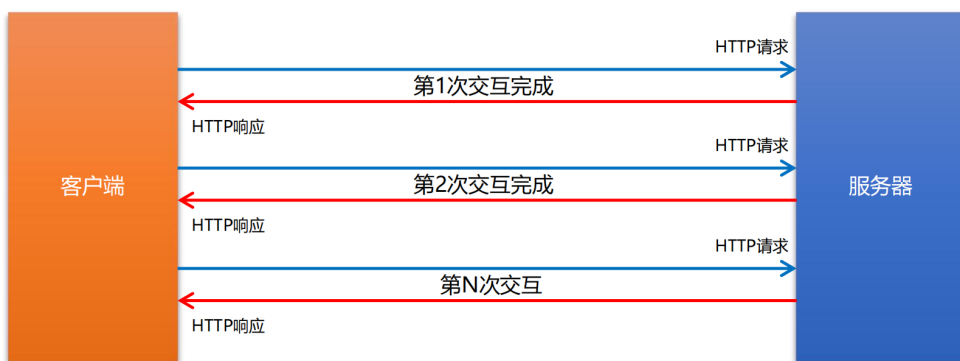
## 5.1.5 什么是HTTP协议

// HTTP 协议即超文本传送协议（HyperText Transfer Protocol），它规定了客户端与服务器之间进行网页内容传输时，所必须遵守的传输格式。

// 客户端要以HTTP协议要求的格式把数据提交到服务器  
// 服务器要以HTTP协议要求的格式把内容响应给客户端

## 5.1.6 HTTP协议的交互模型

HTTP 协议采用了 **请求/响应** 的交互模型。



## 5.2 HTTP请求消息

### 5.2.1 什么是HTTP请求消息

//由于 HTTP 协议属于客户端浏览器和服务器之间的通信协议。因此，客户端发起的请求叫做 HTTP 请求，客户端发送到服务器的消息，叫做 HTTP 请求消息。

// 注意：HTTP 请求消息又叫做 HTTP 请求报文。

### 5.2.2 HTTP请求消息的组成部分

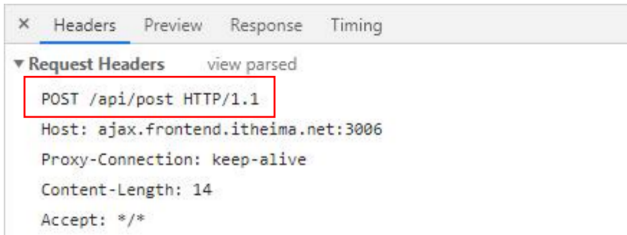
HTTP 请求消息由**请求行**（request line）、**请求头部**（header）、**空行**和**请求体**4个部分组成。



#### 2-1 请求行

请求行由请求方式、URL 和 HTTP 协议版本 3 个部分组成，他们之间使用空格隔开。

请求方式	空格	URL	空格	协议版本	回车符	换行符
------	----	-----	----	------	-----	-----



2-2 请求头部

请求头部用来描述客户端的基本信息，从而把客户端相关的信息告知服务器。比如：User-Agent 用来说明当前是什么类型的浏览器；Content-Type 用来描述发送到服务器的数据格式；Accept 用来描述客户端能够接收什么类型的返回内容；Accept-Language 用来描述客户端期望接收哪种人类语言的文本内容。

请求头部由多行 键/值对 组成，每行的键和值之间用英文的冒号分隔。

头部字段名称	:(冒号)	值	回车符	换行符
.....				
头部字段名称	:(冒号)	值	回车符	换行符

2-2-1 请求头部 – 常见的请求头字段

头部字段	说明
Host	要请求的服务器域名
Connection	客户端与服务器的连接方式(close 或 keepalive)
Content-Length	用来描述请求体的大小
Accept	客户端可识别的响应内容类型列表
User-Agent	产生请求的浏览器类型
Content-Type	客户端告诉服务器实际发送的数据类型
Accept-Encoding	客户端可接收的内容压缩编码形式
Accept-Language	用户期望获得的自然语言的优先顺序

2-3 空行

最后一个请求头字段的后面是一个空行，通知服务器请求头部至此结束。

请求消息中的空行，用来分隔请求头部与请求体。

头部字段名称	:(冒号)	值	回车符	换行符
空行(回车符 或 换行符)				
请求体				

2-4 请求体

请求体中存放的，是要通过 **POST 方式**提交到服务器的数据。

头部字段名称	:(冒号)	值	回车符	换行符
空行(回车符 或 换行符)				
请求体				

**注意：**只有 POST 请求才有请求体，GET 请求没有请求体！

2-5 总结



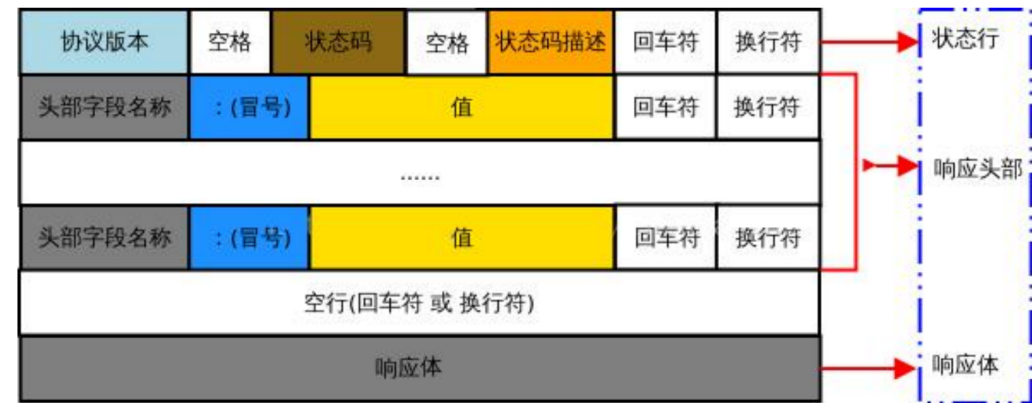
5.3 HTTP响应消息

5.3.1 什么是HTTP响应消息

// 响应消息就是服务器响应给客户端的消息内容，也叫作响应报文。

5.3.2 HTTP响应消息的组成部分

HTTP响应消息由**状态行**、**响应头部**、**空行**和**响应体** 4 个部分组成，如下图所示：



2-1 状态行



**状态行**由 HTTP 协议版本、状态码和状态码的描述文本 3 个部分组成，他们之间使用空格隔开；

协议版本	空格	状态码	空格	状态码描述	回车符	换行符
------	----	-----	----	-------	-----	-----

▼ Response Headers    view parsed

HTTP/1.1 200 OK

X-Powered-By: Express

Access-Control-Allow-Origin: \*

Content-Type: application/json; charset=utf-8

Content-Length: 68

ETag: W/"44-nT/y6yOfj7H40EVW1DWB1MG+Pq0"

Date: Wed, 27 Nov 2019 01:48:57 GMT

Connection: keep-alive

## 2-2 响应头部

**响应头部**用来描述服务器的基本信息。响应头部由多行 键/值对 组成，每行的键和值之间用英文的冒号分隔。

头部字段名称	:(冒号)	值	回车符	换行符
.....				
头部字段名称	:(冒号)	值	回车符	换行符

### 2-2-1 响应头部 - 常见的响应头字段

▼ Response Headers    view parsed

HTTP/1.1 200 OK

X-Powered-By: Express

Access-Control-Allow-Origin: \*

Content-Type: application/json; charset=utf-8

Content-Length: 68

ETag: W/"44-nT/y6yOfj7H40EVW1DWB1MG+Pq0"

Date: Wed, 27 Nov 2019 02:13:24 GMT

Connection: keep-alive

## 2-3 空行

在最后一个响应头部字段结束之后，会紧跟一个**空行**，用来通知客户端**响应头部至此结束**。

响应消息中的空行，用来分隔**响应头部**与**响应体**。

头部字段名称	:(冒号)	值	回车符	换行符
空行(回车符 或 换行符)				
响应体				

## 2-4 响应体

响应体中存放的，是服务器响应给客户端的资源内容。

头部字段名称	: (冒号)	值	回车符	换行符
空行(回车符 或 换行符)				
响应体				

HeadersPreviewResponseTiming

```
{"message": "POST请求测试成功", "data": {"name": "zs", "age": "20"}}
```

2-5 总结

5. 总结



5.4 HTTP请求方法

5.4.1 什么是HTTP请求方法

// HTTP 请求方法，属于 HTTP 协议中的一部分，请求方法的作用是：用来表明要对服务器上的资源执行的操作。最常用的请求方法是 GET 和 POST。

5.4.2 HTTP的请求方法

序号	方法	描述
1	GET	(查询)发送请求来获得服务器上的资源，请求体中不会包含请求数据，请求数据放在协议头中。
2	POST	(新增)向服务器提交资源（例如提交表单或上传文件）。数据被包含在请求体中提交给服务器。
3	PUT	(修改)向服务器提交资源，并使用提交的新资源，替换掉服务器对应的旧资源。
4	DELETE	(删除)请求服务器删除指定的资源。
5	HEAD	HEAD 方法请求一个与 GET 请求的响应相同的响应，但没有响应体。
6	OPTIONS	获取http服务器支持的http请求方法，允许客户端查看服务器的性能，比如ajax跨域时的预检等。
7	CONNECT	建立一个到由目标资源标识的服务器的隧道。
8	TRACE	沿着到目标资源的路径执行一个消息环回测试，主要用于测试或诊断。
9	PATCH	是对 PUT 方法的补充，用来对已知资源进行局部更新。

## 5.5 HTTP响应状态代码

### 5.5.1 什么是HTTP响应状态码

//HTTP 响应状态码（HTTP Status Code），也属于 HTTP 协议的一部分，用来标识响应的状态。

//响应状态码会随着响应消息一起被发送至客户端浏览器，浏览器根据服务器返回的响应状态码，就能知道这次HTTP 请求的结果是成功还是失败了。

### 5.5.2 HTTP响应状态码的组成及分类

// HTTP 状态码由三个十进制数字组成，第一个十进制数字定义了状态码的类型，后两个数字用来对状态码进行细分。

// HTTP 状态码共分为 5 种类型：

分类	分类描述
1**	信息，服务器收到请求，需要请求者继续执行操作（实际开发中很少遇到 1** 类型的状态码）
2**	成功，操作被成功接收并处理
3**	重定向，需要进一步的操作以完成请求
4**	客户端错误，请求包含语法错误或无法完成请求
5**	服务器错误，服务器在处理请求的过程中发生了错误

### 5.5.3 常见的HTTP响应状态码

#### 3-1 2\*\*成功相关的响应状态码

2\*\* 范围的状态码，表示服务器已成功接收到请求并进行处理。常见的 2\*\* 类型的状态码如下：

状态码	状态码英文名称	中文描述
200	OK	请求成功。一般用于 GET 与 POST 请求
201	Created	已创建。成功请求并创建了新的资源，通常用于 POST 或 PUT 请求

#### 3-2 3\*\* 重定向相关的响应状态码

3\*\* 范围的状态码，表示表示服务器要求客户端重定向，需要客户端进一步的操作以完成资源的请求。常见的 3\*\* 类型的状态码如下：

状态码	状态码英文名称	中文描述
301	Moved Permanently	永久移动。请求的资源已被永久的移动到新URI，返回信息会包括新的URI，浏览器会自动定向到新URI。今后任何新的请求都应使用新的URI代替
302	Found	临时移动。与301类似。但资源只是临时被移动。客户端应继续使用原有URI
304	Not Modified	未修改。所请求的资源未修改，服务器返回此状态码时，不会返回任何资源（响应消息中不包含响应体）。客户端通常会缓存访问过的资源。

#### 3-3 4\*\* 客户端错误相关的响应状态码

4\*\* 范围的状态码，表示客户端的请求有非法内容，从而导致这次请求失败。常见的 4\*\* 类型的状态码如下：

状态码	状态码英文名称	中文描述
400	Bad Request	1、语义有误，当前请求无法被服务器理解。除非进行修改，否则客户端不应该重复提交这个请求。 2、请求参数有误。
401	Unauthorized	当前请求需要用户验证。
403	Forbidden	服务器已经理解请求，但是拒绝执行它。
404	Not Found	服务器无法根据客户端的请求找到资源（网页）。
408	Request Timeout	请求超时。服务器等待客户端发送的请求时间过长，超时。

### 3-4 5\*\* 服务端错误相关的响应状态码

5\*\* 范围的状态码，表示服务器未能正常处理客户端的请求而出现意外错误。常见的 5\*\* 类型的状态码如下：

状态码	状态码英文名称	中文描述
500	Internal Server Error	服务器内部错误，无法完成请求。
501	Not Implemented	服务器不支持该请求方法，无法完成请求。只有 GET 和 HEAD 请求方法是要求每个服务器必须支持的，其它请求方法在不支持的服务器上会返回501
503	Service Unavailable	由于超载或系统维护，服务器暂时的无法处理客户端的请求。

## 四，promise

### 1. Promise是什么？

// **Promise** 是异步编程的一种解决方案，比传统的解决方案回调函数，更合理和更强大。**ES6** 将其写进了语言标准，统一了用法，原生提供了**Promise**对象。

//指定回调函数方式更灵活易懂。  
// 解决异步 \*\*回调地狱\*\* 的问题。

### 回调地狱

- 当一个回调函数嵌套一个回调函数的时候
- 就会出现一个嵌套结构
- 当嵌套的多了就会出现回调地狱的情况
- 比如我们发送三个 ajax 请求
  - 第一个正常发送
  - 第二个请求需要第一个请求的结果中的某一个值作为参数
  - 第三个请求需要第二个请求的结果中的某一个值作为参数

```
ajax({
 url: '我是第一个请求',
 success (res) {
 // 现在发送第二个请求
 ajax({
 url: '我是第二个请求',
 data: { a: res.a, b: res.b },
 success (res2) {
```

```

// 进行第三个请求
ajax({
 url: '我是第三个请求',
 data: { a: res2.a, b: res2.b },
 success (res3) {
 console.log(res3)
 }
})
}
})
}
})
}
})

```

- 回调地狱，其实就是回调函数嵌套过多导致的



```

1 function hell(win) {
2 // for listener purpose
3 return function() {
4 loadLink(win, REMOTE_SRC+'assets/css/style.css', function() {
5 loadLink(win, REMOTE_SRC+'lib/async.js', function() {
6 loadLink(win, REMOTE_SRC+'lib/easyXDM.js', function() {
7 loadLink(win, REMOTE_SRC+'lib/json2.js', function() {
8 loadLink(win, REMOTE_SRC+'lib/underscore.min.js', function() {
9 loadLink(win, REMOTE_SRC+'lib/backbone.min.js', function() {
10 loadLink(win, REMOTE_SRC+'dev/base_dev.js', function() {
11 loadLink(win, REMOTE_SRC+'assets/js/deps.js', function() {
12 loadLink(win, REMOTE_SRC+'src/' + win.loader_path + '/loader.js', function() {
13 async.eachSeries(SRIPTS, function(src, callback) {
14 loadScript(win, BASE_URL+src, callback);
15 });
16 });
17 });
18 });
19 });
20 });
21 });
22 });
23 });
24 });
25 });
26 }

```

- 当代码成为这个结构以后，已经没有维护的可能了

## 2. Promise使用

- 语法:

```

new Promise(function (resolve, reject) {
 // resolve 表示成功的回调
 // reject 表示失败的回调
}).then(function (res) {
 // 成功的函数
}).catch(function (err) {
 // 失败的函数
})

```

## 3. Promise 对象的状态

Promise 对象通过自身的状态，来控制异步操作。Promise 实例具有三种状态。

异步操作未完成（**pending**）  
异步操作成功（**fulfilled**）  
异步操作失败（**rejected**）

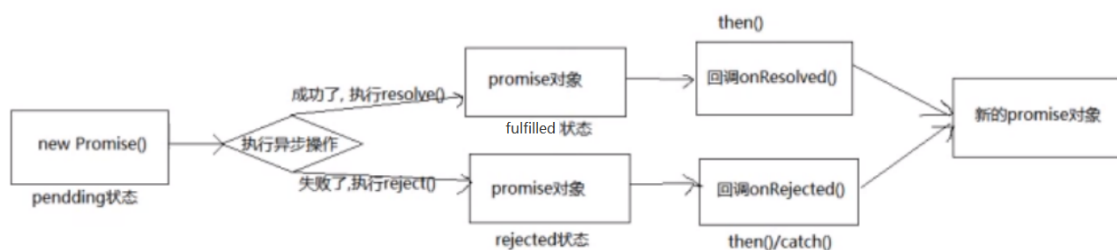
这三种的状态的变化途径只有两种。

从“未完成”到“成功”  
从“未完成”到“失败”

一旦状态发生变化，就凝固了，不会再有新的状态变化。这也是 Promise 这个名字的由来，它的英语意思是“承诺”，一旦承诺成效，就不得再改变了。这也意味着，Promise 实例的状态变化只可能发生一次。

因此，Promise 的最终结果只有两种。

异步操作成功，Promise 实例传回一个值（**value**），状态变为**fulfilled**。  
异步操作失败，Promise 实例抛出一个错误（**error**），状态变为**rejected**。



## 4.Promise对象方法

Promise 是一个对象，也是一个构造函数。

### 4.1.Promise.resolve

将现有对象转为 Promise 对象

```
Promise.resolve('kerwin')
// 等价于
new Promise(resolve => resolve('kerwin'))
```

### 4.2.Promise.reject

`Promise.reject(reason)` 方法也会返回一个新的 Promise 实例，该实例的状态为 `rejected`。

```
const p = Promise.reject('error');
// 等同于
const p = new Promise((resolve, reject) => reject('error'))
```

## 4.3.Promise.all

`Promise.all()` 方法用于将多个 Promise 实例，包装成一个新的 Promise 实例。

```
const p = Promise.all([p1, p2, p3]);
```

p 的状态由 p1, p2, p3 决定，分成两种情况。

(1) 只有 p1、p2、p3 的状态都变成 fulfilled，p 的状态才会变成 fulfilled，此时 p1、p2、p3 的返回值组成一个数组，传递给 p 的回调函数。

(2) 只要 p1、p2、p3 之中有一个被 rejected，p 的状态就变成 rejected，此时第一个被 reject 的实例的返回值，会传递给 p 的回调函数。

## 4.4.Promise.race

`Promise.race()` 方法同样是将多个 Promise 实例，包装成一个新的 Promise 实例。

```
const p = Promise.race([p1, p2, p3]);
```

上面代码中，只要 p1、p2、p3 之中有一个实例率先改变状态，p 的状态就跟着改变。那个率先改变的 Promise 实例的返回值，就传递给 p 的回调函数。

## 4.5.Promise.allSettled

`Promise.allSettled()` 方法，用来确定一组异步操作是否都结束了（不管成功或失败）。所以，它的名字叫做“Settled”，包含了“fulfilled”和“rejected”两种情况。

```
const promises = [ajax('/200接口'), ajax('/401接口')];

Promise.allSettled(promises).then(results=>{
 // 过滤出成功的请求
 results.filter(item => item.status === 'fulfilled');
 过滤出失败的请求
 results.filter(item=> item.status === 'rejected');
})
```

## 4.6.Promise.any

只要参数实例有一个变成 fulfilled 状态，包装实例就会变成 fulfilled 状态；如果所有参数实例都变成 rejected 状态，包装实例就会变成 rejected 状态。

`Promise.any()` 跟 `Promise.race()` 方法很像，只有一点不同，就是 `Promise.any()` 不会因为某个 Promise 变成 rejected 状态而结束，必须等到所有参数 Promise 变成 rejected 状态才会结束。

## 5.手写Promise

```
/*
 * @作者: kerwin
 */
function KerwinPromise(executor) {
 this.status = "pending";
```



```

 this.result = undefined;
 this.cb = [];
 var _this = this;

 function resolve(res) {
 if (_this.status !== "pending") return;
 // console.log(_this)
 _this.status = "fulfilled"
 _this.result = res;

 _this.cb.forEach(item => {
 item.successCB && item.successCB(_this.result)
 });
 }

 function reject(res) {
 if (_this.status !== "pending") return;
 // console.log("reject")
 _this.status = "rejected"
 _this.result = res;
 _this.cb.forEach(item => {
 item.failCB && item.failCB(_this.result)
 });
 }
 executor(resolve, reject)
}

KerwinPromise.prototype.then = function (successCB, failCB) {

 if(!successCB){
 successCB = value=>value
 }
 if(!failCB){
 failCB = error=>error
 }

 // successCB()
 return new KerwinPromise((resolve, reject) => {
 if (this.status === "fulfilled") {
 var result = successCB && successCB(this.result)
 // console.log(result);

 if (result instanceof KerwinPromise) {
 result.then(res => {
 // console.log(res)
 resolve(res);
 }, err => {
 // console.log(err)
 reject(err)
 })
 } else {
 resolve(result);
 }
 }
 if (this.status === "rejected") {
 var result = failCB && failCB(this.result)

 if (result instanceof KerwinPromise) {

```



```

 result.then(res => {
 // console.log(res)
 resolve(res);
 }, err => {
 // console.log(err)
 reject(err)
 })
 } else {
 reject(result);
 }
}

if (this.status === "pending") {
 //收集回调
 this.cb.push({
 successCB: () => {
 var result = successCB && successCB(this.result)

 if (result instanceof KerwinPromise) {
 result.then(res => {
 // console.log(res)
 resolve(res);
 }, err => {
 // console.log(err)
 reject(err)
 })
 } else {
 resolve(result);
 }
 },
 failCB: () => {
 var result = failCB && failCB(this.result)
 if (result instanceof KerwinPromise) {
 result.then(res => {
 // console.log(res)
 resolve(res);
 }, err => {
 // console.log(err)
 reject(err)
 })
 } else {
 reject(result);
 }
 }
 })
}

})
}

KerwinPromise.prototype.catch= function(failCB){
 this.then(undefined,failCB)
}

```

## 6.Async与Await

## 6.1.Async

async 函数，使得异步操作变得更加方便。

- 更好的语义。
- 返回值是 Promise。

```
async function test(){

}
test()
```

## 6.2.Await

await 命令后面是一个 Promise 对象，返回该对象的结果。如果不是 Promise 对象，就直接返回对应的值。

```
async function test(){
 var res1 = await ajax("http://localhost:3000/news1")
 var res2 = await ajax("http://localhost:3000/news2")
 return res2
}

test().then(res=>{
 console.log("返回结果",res)
}).catch(err=>{
 console.log("err",err)
})
```

## 6.3.错误处理

```
try{
 var res1 = await ajax("http://localhost:3000/news1")
 var res2 = await ajax("http://localhost:3000/news2")
}catch(err){
 console.log("err",err)
}
```

# 五, npm

<https://www.npmjs.com/>

## 1, npm基础知识

```
// 从 https://www.npmjs.com/ 网站上搜索自己所需要的包
// 从 https://registry.npmjs.org/ 服务器上下载自己需要的包

// 初次装包完成后，在项目文件夹下多一个叫做 node_modules 的文件夹和 package-lock.json 的配置文件。
// 其中：
```

// **node\_modules** 文件夹用来存放所有已安装到项目中的包。**require()** 导入第三方包时，就是从这个目录中查找并加载包。

// **package-lock.json** 配置文件用来记录 **node\_modules** 目录下的每一个包的下载信息，例如包的名字、版本号、下载地址等。

// npm指令

1, **npm -v** // 查看版本号

2, **npm init -y** // 快速初始化项目，快速创建 **package.json** 这个包管理配置文件

// 上述命令只能在英文的目录下成功运行！所以，项目文件夹的名称一定要使用英文命名，不要使用中文，不能出现空格

// 如果某些包只在项目开发阶段会用到，在项目上线之后不会用到，则建议把这些包记录到 **devDependencies** 节点中

3, **npm i 包名 -D** || **npm i 包名 --save-dev**

// 查看当前的下包镜像源

4, **npm config get registry**

// 将下包的镜像源换为淘宝镜像源

5, **npm config set registry = https://registry.npm.taobao.org/**

// 为了更方便的切换下包的镜像源，我们可以安装 **nrm** 这个小工具，利用 **nrm** 提供的终端命令，可以快速查看和切换下包的镜像源

6, **npm i nrm -g**

// 查看所有可用的镜像源

7, **nrm ls**

// 将下载镜像源转换为淘宝镜像源

8, **nrm use taobao**

// 开发依赖包

9 **npm i 包名 -D** // 被记录到 **devDependencies** 节点中的包，只在开发期间会用到

// 核心依赖包

10 **npm i 包名** // 被记录到 **dependencies** 节点中的包，在开发期间和项目上线之后都会用到

## 2, i5ting\_toc

i5ting\_toc 是一个可以把 md 文档转为 html 页面的小工具，使用步骤如下：

// 安装

**npm install -g i5ting\_toc**

// 把md文档转换为网页

**i5ting\_toc -f 需要转换的md路径 -o**

## 3, npm 查看命令 (以axios为例)

// 查看项目中依赖所在的目录

**npm root**

// 查看全局安装的依赖所在目录

**npm root -g**

// 查看已安装依赖的列表

```
npm list
npm ls

// 查看axios最新的版本号
npm view axios version

// 查看全部axios历史版本号
npm view axios versions

// 查看最新的axios版本的信息
npm view axios
npm info axios

// 查看本地已安装的axios的详细信息
npm list axios
npm ls axios
```

## 4, npm其他命令 (以axios为例)

```
// 清除npm的缓存
npm cache clean

// 清除项目中没有被使用的依赖
npm prune

// 检查依赖是否已经弃用
npm outdated

// 打开默认浏览器跳转到github中axios的页面
npm repo axios

// 打开默认浏览器跳转到github中axios的README.MD文件
npm docs axios
```

# 六, Git

```
// Git 中的三个区域-----工作区、暂存区、Git 仓库

// 工作区 ----- 处理工作的区域
// 暂存区 ----- 已完成的工作的临时存放区域，等待被提交
// Git 仓库 ----- 最终的存放区域

// git中的三种状态 -----已暂存 staged || 已修改 modified || 已提交
committed

// 工作区的文件被修改了，但还没有放到暂存区，就是已修改状态。
// 如果文件已修改并放入暂存区，就属于已暂存状态。
// 如果 Git 仓库中保存着特定版本的文件，就属于已提交状态。
```

## 1,安装并配置 Git

## 1.1 配置用户信息

// 安装完 Git 之后，要做的第一件事就是设置自己的用户名和邮件地址。因为通过 Git 对项目进行版本管理的时候，Git 需要使用这些基本信息，来记录是谁对项目进行了操作：

```
git config --global user.name "aini"
git config --global user.email "ai_aini2022@163.com"
```

## 1.2 Git 的全局配置文件

// 通过 git config --global user.name 和 git config --global user.email 配置的用户名和邮箱地址，会被写入到 C:/Users/用户名文件夹/.gitconfig 文件中。这个文件是 Git 的全局配置文件，配置一次即可永久生效。

## 1.3 检查配置信息

```
// 查看所有的全局配置项
git config --list --global
```

```
// 查看指定的全局配置项
git config user.name
git config user.email
```

## 1.4 获取帮助信息

```
// 查看 git config 命令的帮助手册
git config -help
// 获取 git config 命令的快速参考
git config -h
```

# 2 Git 的基本操作

## 2.1 获取 Git 仓库的两种方式

```
// ① 将尚未进行版本控制的本地目录转换为 Git 仓库
// ② 从其它服务器克隆一个已存在的 Git 仓库
```

## 2.2 在现有目录中初始化仓库

// 如果自己有一个尚未进行版本控制的项目目录，想要用 Git 来控制它，需要执行如下两个步骤：

```
// 在项目目录中，通过鼠标右键打开“Git Bash”
// 执行 git init 命令将当前的目录转化为 Git 仓库
```

// git init 命令会创建一个名为 .git 的隐藏目录，这个 .git 目录就是当前项目的 Git 仓库，里面包含了初始的必要文件，这些文件是 Git 仓库的必要组成部分。

## 2.3 工作区中文件的 4 种状态

```
// 未跟踪 (Untracked) ----- 不被 Git 所管理的文件
// 未修改 (Unmodified) ----- 工作区中文件的内容和 Git 仓库中文件的内容保持一致
// 已修改 (Modified) ----- 工作区中文件的内容和 Git 仓库中文件的内容不一致
// 已暂存 (Staged) ----- 工作区中被修改的文件已被放到暂存区，准备将修改后的文件保
存到 Git 仓库中
```

```
// Git 操作的终极结果：让工作区中的文件都处于“未修改”的状态
```

## 2.4 检查文件的状态

```
git status
// 在状态报告中可以看到新建的 index.html 文件出现在 Untracked files (未跟踪的文件) 下面。
// 未跟踪的文件意味着 Git 在之前的快照 (提交) 中没有这些文件；Git 不会自动将之纳入跟踪范围，除
非明确地告诉它“我需要使用 Git 跟踪管理该文件”

// 精简的方式显示文件状态
git status -s || git status --short

// 未跟踪文件前面有红色的 ?? 标记
```

## 2.5 跟踪新文件

```
// 跟踪单个文件
git add index.html
// 跟踪所有文件
git add .

// 此时再运行 git status 命令，会看到 index.html 文件在 Changes to be committed 这行的
下面，说明已被跟踪，并处于暂存状态：
// 已跟踪的文件绿色显示
```

## 2.6 提交更新

```
// 现在暂存区中有一个 index.html 文件等待被提交到 Git 仓库中进行保存。可以执行 git commit
命令进行提交，其中 -m 选项后面是本次的提交消息，用来对提交的内容做进一步的描述：

git commit -m "提交信息内容"

// 提交成功之后，再次检查文件的状态，得到提示如下：nothing to commit, working tree clean
// 证明工作区中所有的文件都处于“未修改”的状态，没有任何文件需要被提交。
```

## 2.7 对已提交的文件进行修改

// 目前, `index.html` 文件已经被 `Git` 跟踪, 并且工作区和 `Git` 仓库中的 `index.html` 文件内容保持一致。当我们修改了工作区中 `index.html` 的内容之后, 再次运行 `git status` 和 `git status -s` 命令, 会看到如下的内容:

// 文件 `index.html` 出现在 `Changes not staged for commit` 这行下面, 说明已跟踪文件的内容发生了变化, 但还没有放到暂存区。

// 注意: 修改过的、没有放入暂存区的文件前面有红色的 `M` 标记

```
C:\Windows\System32\cmd.exe
E:\code>git status
On branch master
Changes not staged for commit:
 (use "git add <file>..." to update what will be committed)
 (use "git checkout -- <file>..." to discard changes in working directory)

 modified: index.html

no changes added to commit (use "git add" and/or "git commit -a")
E:\code>git status -s
M index.html
```

## 2.8 暂存已修改的文件

// 目前, 工作区中的 `index.html` 文件已被修改, 如果要暂存这次修改, 需要再次运行 `git add` 命令, 这个命令是个多功能的命令, 主要有如下 3 个功效:

- 1, 可以用它开始跟踪新文件
- 2, 把已跟踪的、且已修改的文件放到暂存区
- 3, 把有冲突的文件标记为已解决状态

```
C:\Windows\System32\cmd.exe
E:\code>git add index.html 把已修改的文件放入暂存区
E:\code>git status 查看详细的文件状态报告
On branch master
Changes to be committed:
 (use "git reset HEAD <file>..." to unstage)

 modified: index.html

E:\code>git status -s 查看精简的文件状态报告,
M index.html 绿色的 M 表示文件已修改且已放入暂存区
```

## 2.9 提交已暂存的文件

// 再次运行 `git commit -m "提交消息"` 命令, 即可将暂存区中记录的 `index.html` 的快照, 提交到 `Git` 仓库中进行保存:

```
C:\Windows\System32\cmd.exe
E:\code>git commit -m "初始化了index.html中的内容" 将暂存区中的文件提交到 Git 仓库
[master 554647a] 初始化了index.html中的内容
 1 file changed, 14 insertions(+)
E:\code>git status 检查工作区中文件的状态
On branch master
nothing to commit, working tree clean
```

## 2.10 撤销对文件的修改

// 撤销对文件的修改指的是：把对工作区中对应文件的修改，还原成 **Git** 仓库中所保存的版本。  
// 操作的结果：所有的修改会丢失，且无法恢复！危险性比较高，请慎重操作！  
// 撤销操作的本质：用 **Git** 仓库中保存的文件，覆盖工作区中指定的文件。

```
git checkout --index.html // 撤销对index.html的修改
```

## 2.11 向暂存区中一次性添加多个文件

```
git add .
```

// 今后在项目开发中，会经常使用这个命令，将新增和修改过后的文件加入暂存区。

## 2.12 取消暂存的文件

// 如果需要从暂存区中移除对应的文件，可以使用如下的命令：

```
git reset HEAD 要移除的文件名
```

## 2.13 跳过使用暂存区域

// **Git** 标准的工作流程是工作区 → 暂存区 → **Git** 仓库，但有时候这么做略显繁琐，此时可以跳过暂存区，直接将工作区中的修改提交到 **Git** 仓库，这时候 **Git** 工作的流程简化为了工作区 → **Git** 仓库。

// **Git** 提供了一个跳过使用暂存区域的方式，只要在提交的时候，给 `git commit` 加上 `-a` 选项，**Git** 就会自动把所有已经跟踪过的文件暂存起来一并提交，从而跳过 `git add` 步骤：

```
git commit -a -m "提交信息内容"
```

## 2.14 移除文件

// 从 **Git** 仓库中移除文件的方式有两种：

// 从 **Git** 仓库和工作区中同时移除指定的文件

```
git rm -f index.html
```

// 只从 **Git** 仓库中移除指定的文件，但保留工作区中对应的文件

```
git rm --cached index.css
```

## 2.15 忽略文件

// 一般我们总会有些文件无需纳入 **Git** 的管理，也不希望它们总出现在未跟踪文件列表。在这种情况下，我们可以创建一个名为 `.gitignore` 的配置文件，列出要忽略的文件的匹配模式。

// 文件 `.gitignore` 的格式规范如下：

- 1，以 `#` 开头的是注释
- 2，以 `/` 结尾的是目录
- 3，以 `/` 开头防止递归
- 4，以 `!` 开头表示取反
- 5，可以使用 `glob` 模式进行文件和文件夹的匹配（`glob` 指简化了的正则表达式）



## 2.16 glob 模式

所谓的 **glob 模式**是指简化了的正则表达式：

- ① **星号 \*** 匹配零个或多个任意字符
- ② **[abc]** 匹配任何一个列在方括号中的字符（此案例匹配一个 a 或匹配一个 b 或匹配一个 c）
- ③ **问号 ?** 只匹配一个任意字符
- ④ 在方括号中使用**短划线**分隔两个字符，表示所有在这两个字符范围内的都可以匹配（比如 [0-9] 表示匹配所有 0 到 9 的数字）
- ⑤ **两个星号 \*\*** 表示匹配任意中间目录（比如 a/\*\*/z 可以匹配 a/z 、 a/b/z 或 a/b/c/z 等）

## 2.17 .gitignore 文件的例子

```
// 忽略所有的 .a 文件、
*.a

// 但跟中所有的 lib.a 即便你在前面忽略了 .a 文件
!lib.a

// 只忽略当前目录下的 TODO 文件，而不忽略 subdir/TODO
/TODO

// 忽略任何目录下名为build的文件夹
build/

// 忽略 doc/notes.txt 但不忽略 doc/server/arch.txt
doc/*.txt

// 忽略 doc/ 目录及其所有子目录下的.pdf 文件
doc/**/*.*pdf
```

```
1 # 忽略所有的 .a 文件
2 *.a
3
4 # 但跟踪所有的 lib.a, 即便你在前面忽略了 .a 文件
5 !lib.a
6
7 # 只忽略当前目录下的 TODO 文件, 而不忽略 subdir/TODO
8 /TODO
9
10 # 忽略任何目录下名为 build 的文件夹
11 build/
12
13 # 忽略 doc/notes.txt, 但不忽略 doc/server/arch.txt
14 doc/*.txt
15
16 # 忽略 doc/ 目录及其所有子目录下的 .pdf 文件
17 doc/**/*.*pdf
```

## 2.18 查看提交历史

```
1 # 按时间先后顺序列出所有的提交历史, 最近的提交排在最上面
2 git log
3
4 # 只展示最新的两条提交历史, 数字可以按需进行填写
5 git log -2
6
7 # 在一行上展示最近两条提交历史的信息
8 git log -2 --pretty=oneline
9
10 # 在一行上展示最近两条提交历史的信息, 并自定义输出的格式
11 # %h 提交的简写哈希值 %an作者名字 %ar作者修订日期, 按多久以前的方式显示 %s提交说明
12 git log -2 --pretty=format:"%h | %an | %ar | %s"
```

## 2.19 回退到指定的版本

```
1 # 在一行上展示所有的提交历史
2 git log --pretty=oneline
3
4 # 使用 git reset --hard 命令, 根据指定的提交 ID 回退到指定版本
5 git reset --hard <CommitID>
6
7 # 在旧版本中使用 git reflog --pretty=oneline 命令, 查看命令操作的历史
8 git reflog --pretty=oneline
9
10 # 再次根据最新的提交 ID, 跳转到最新的版本
11 git reset --hard <CommitID>
```

## 3, Github - 远程仓库的使用

### 3.1, 远程仓库的两种访问方式

// Github 上的远程仓库, 有两种访问方式, 分别是 HTTPS 和 SSH。它们的区别是:

- 1, HTTPS: 零配置; 但是每次访问仓库时, 需要重复输入 Github 的账号和密码才能访问成功
- 2, SSH: 需要进行额外的配置; 但是配置成功后, 每次访问仓库时, 不需重复输入 Github 的账号和密码

// 注意: 在实际开发中, 推荐使用 SSH 的方式访问远程仓库。

### 3.2 ,基于 HTTPS 将本地仓库上传到 Github

Quick setup — if you've done this kind of thing before

☐ Set up in Desktop or ☒ HTTPS ☐ SSH

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

**1 本地没有现成的 Git 仓库**

...or create a new repository on the command line

```
echo "# project 02" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin https://github.com/teacher-liu/project_02.git
git push -u origin master
```

1. 使用终端命令创建 README.md 文档, 并写入初始内容为 #project 02

2. 初始化本地 Git 仓库, 并将文件的修改提交到本地的 Git 仓库中

3. 将本地仓库和远程仓库进行关联, 并把远程仓库命名为 origin

4. 将本地仓库中的内容推送到远程的 origin 仓库中

**2 本地有现成的 Git 仓库**

...or push an existing repository from the command line

```
git remote add origin https://github.com/teacher-liu/project_02.git
git push -u origin master
```

1. 将本地仓库和远程仓库进行关联, 并把远程仓库命名为 origin

2. 将本地仓库中的内容推送到远程的 origin 仓库中

### 3.3 SSH key

```
// SSH key 的作用：实现本地仓库和 Github 之间免登录的加密数据传输。
// SSH key 的好处：免登录身份认证、数据加密传输。

// SSH key 由两部分组成，分别是：
1, id_rsa（私钥文件，存放于客户端的电脑中即可）
2, id_rsa.pub（公钥文件，需要配置到 Github 中
```

### 3.4 生成 SSH key

```
// 打开 Git Bash

// 粘贴如下的命令，并将 your_email@example.com 替换为注册 Github 账号时填写的邮箱：
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"

// 连续敲击 3 次回车，即可在 C:\Users\用户名文件夹\.ssh 目录中生成 id_rsa 和 id_rsa.pub 两个文件
```

### 3.5 配置 SSH key

```
// 使用记事本打开 id_rsa.pub 文件，复制里面的文本内容
1, 在浏览器中登录 Github，点击头像 -> Settings -> SSH and GPG Keys -> New SSH key
2, 将 id_rsa.pub 文件中的内容，粘贴到 Key 对应的文本框中
3, 在 Title 文本框中任意填写一个名称，来标识这个 key 从何而来
```

- ① 使用记事本打开 `id_rsa.pub` 文件，复制里面的文本内容
- ② 在浏览器中登录 Github，点击头像 -> Settings -> SSH and GPG Keys -> New SSH key
- ③ 将 `id_rsa.pub` 文件中的内容，粘贴到 Key 对应的文本框中
- ④ 在 Title 文本框中任意填写一个名称，来标识这个 Key 从何而来

### 3.6 检测 Github 的 SSH key 是否配置成功

打开 Git Bash，输入如下的命令并回车执行：

```
1 ssh -T git@github.com
```

上述的命令执行成功后，可能会看到如下的提示消息：

```
1 The authenticity of host 'github.com (IP ADDRESS)' can't be established.
2 RSA key fingerprint is SHA256:nThbg6kXUpJWGl7E1IGOCspRomTxdCARLviKw6E5SY8.
3 Are you sure you want to continue connecting (yes/no)?
```

输入 yes 之后，如果能看到类似于下面的提示消息，证明 SSH key 已经配置成功了：

```
1 Hi username! You've successfully authenticated, but GitHub does not
2 provide shell access.
```

## 3.7 基于 SSH 将本地仓库上传到 Github

Quick setup — if you've done this kind of thing before

Set up in Desktop or **SSH** `git@github.com:teacher-liu/project_03.git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# project_03" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin git@github.com:teacher-liu/project_03.git
git push -u origin master
```

将本地现成的仓库推送到 Github

...or push an existing repository from the command line

```
git remote add origin git@github.com:teacher-liu/project_03.git
git push -u origin master
```

1. 将本地仓库和远程仓库进行关联，并把远程仓库命名为 @origin  
2. 将本地仓库中的内容推送到远程的 origin 仓库中

## 3.8 将远程仓库克隆到本地

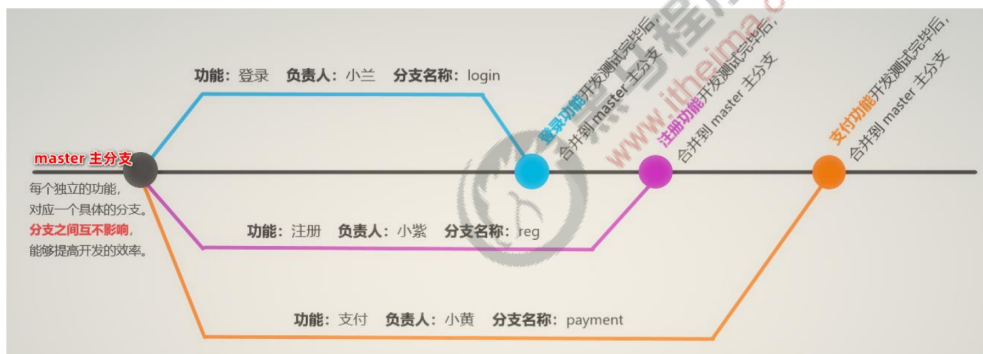
打开 Git Bash，输入如下的命令并回车执行：

```
1 git clone 远程仓库的地址
```

# 4 本地分支操作

## 4.1 master 主分支

在初始化本地 Git 仓库的时候，Git 默认已经帮我们创建了一个名字叫做 **master** 的分支。通常我们把这个 master 分支叫做**主分支**。



在实际工作中，master 主分支的作用是：**用来保存和记录整个项目已完成的功能代码。**

因此，不允许程序员直接在 master 分支上修改代码，因为这样做的风险太高，容易导致整个项目崩溃。

## 4.2 功能分支

由于程序员不能直接在 master 分支上进行功能的开发，所以就有了**功能分支**的概念。

**功能分支**指的是**专门用来开发新功能的分支**，它是临时从 master 主分支上分叉出来的，当新功能开发且测试完毕后，最终需要合并到 master 主分支上，如图所示：



## 4.3 查看分支列表

```
// 查看分支列表
```

```
git branch
```

```
// 分支名字前面的 * 号表示当前所处的分支。
```

## 4.4 创建新分支

```
// 创建分支
```

```
git branch 分支名称
```

## 4.5 切换分支

```
// 切换分支
```

```
git checkout 分支名称
```

## 4.6 分支的快速创建和切换

```
// -b 表示创建一个新分支
```

```
// checkout 表示切换到新建的分支上
```

```
git checkout -b 分支名称
```

## 4.7 合并分支

```
// 切换到master分支
```

```
git checkout master
```

```
// 将master分支上运行git merge 命令，将login分支的代码合并到master分支
```

```
git merge login
```

## 4.8 删除分支

```
// 删除分支
git branch -d 分支名称
```

## 4.9 遇到冲突时的分支合并

如果在两个不同的分支中，对同一个文件进行了不同的修改，Git 就没法干净的合并它们。此时，我们需要打开这些包含冲突的文件然后**手动解决冲突**。

```
1 # 假设：在把 reg 分支合并到 master 分支期间，代码发生了冲突
2 git checkout master
3 git merge reg
4
5 # 打开包含冲突的文件，手动解决冲突之后，再执行如下的命令
6 git add .
7 git commit -m "解决了分支合并冲突的问题"
```

## 5,远程分支操作

### 5.1 将本地分支推送到远程仓库

如果是**第一次**将本地分支推送到远程仓库，需要运行如下的命令：

```
1 # -u 表示把本地分支和远程分支进行关联，只在第一次推送的时候需要带 -u 参数
2 git push -u 远程仓库的别名 本地分支名称:远程分支名称
3
4 # 实际案例:
5 git push -u origin payment:pay
6
7 # 如果希望远程分支的名称和本地分支名称保持一致，可以对命令进行简化:
8 git push -u origin payment
```

注意：第一次推送分支需要带 **-u 参数**，此后可以直接使用 **git push** 推送代码到远程分支。

```
// 注意：第一次推送分支需要带 -u 参数，此后可以直接使用 git push 推送代码到远程分支。
```

### 5.2 查看远程仓库中所有的分支列表

```
// 查看远程仓库中所有的分支列表
git remote show 远程仓库名称
```



## 5.3 跟踪分支

跟踪分支指的是：从远程仓库中，把远程分支下载到本地仓库中。需要运行的命令如下：

```
1 # 从远程仓库中，把对应的远程分支下载到本地仓库，保持本地分支和远程分支名称相同
2 git checkout 远程分支的名称
3 # 示例：
4 git checkout pay
5
6 # 从远程仓库中，把对应的远程分支下载到本地仓库，并把下载的本地分支进行重命名
7 git checkout -b 本地分支名称 远程仓库名称/远程分支名称
8 # 示例：
9 git checkout -b payment origin/pay
```

## 5.4 拉取远程分支的最新的代码

可以使用如下的命令，把远程分支最新的代码下载到本地对应的分支中：

```
1 # 从远程仓库，拉取当前分支最新的代码，保持当前分支的代码和远程分支代码一致
2 git pull
```

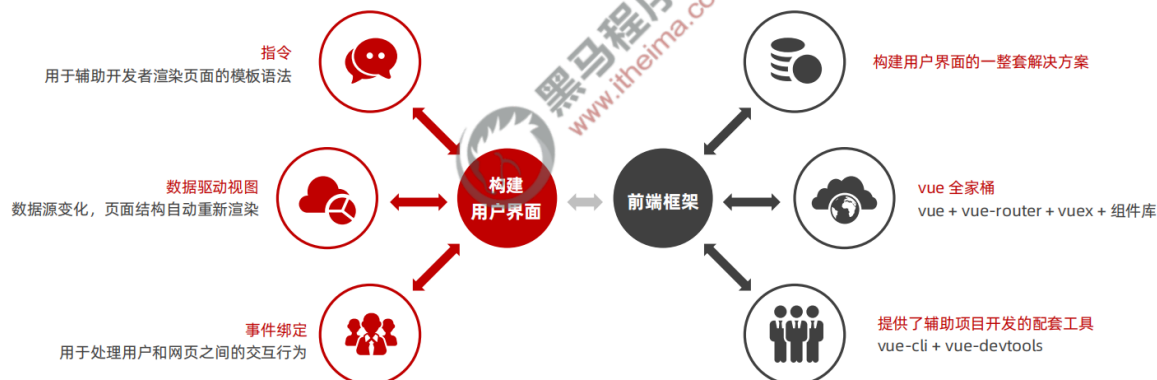
## 5.5 删除远程分支

```
// 删除远程仓库中，指定名称的远程分支
git push 远程仓库名称 --delete 远程分支名称

// 实例
git push origin --delete pay
```

# 六，Vue

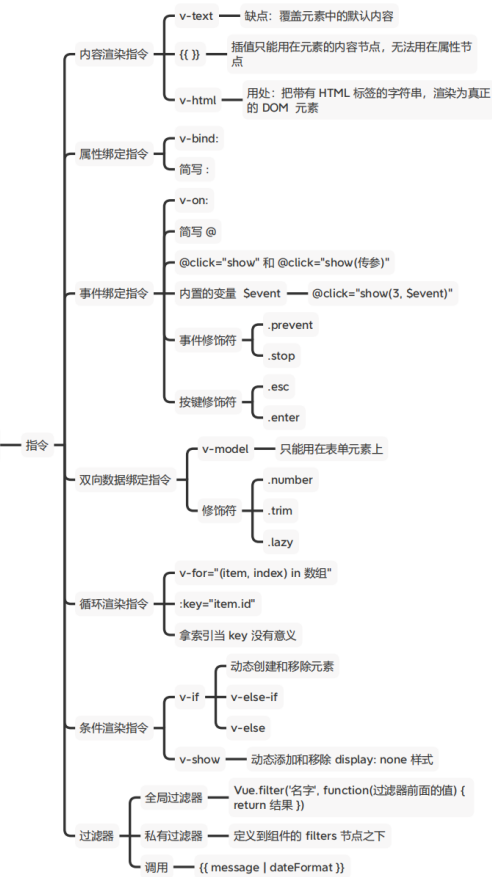
vue 是一套用于构建用户界面的前端框架。



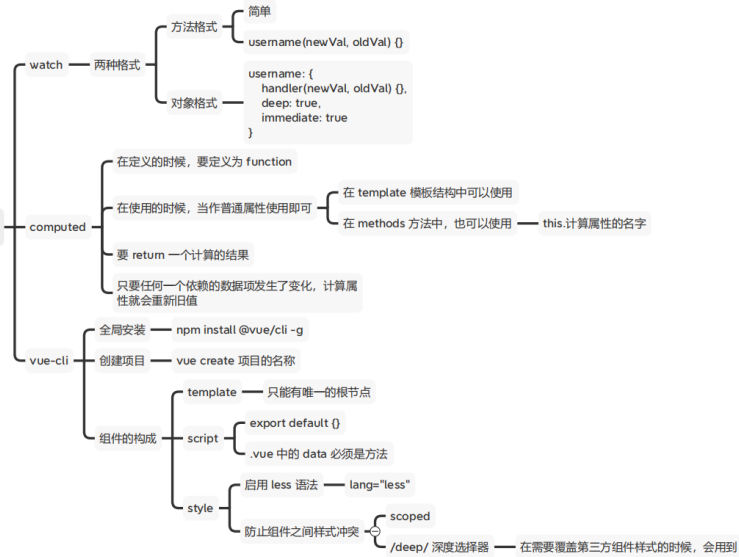


# vue 基础

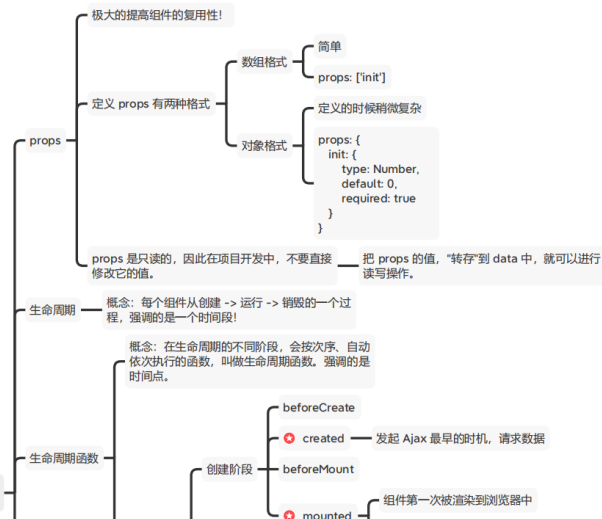
## day1

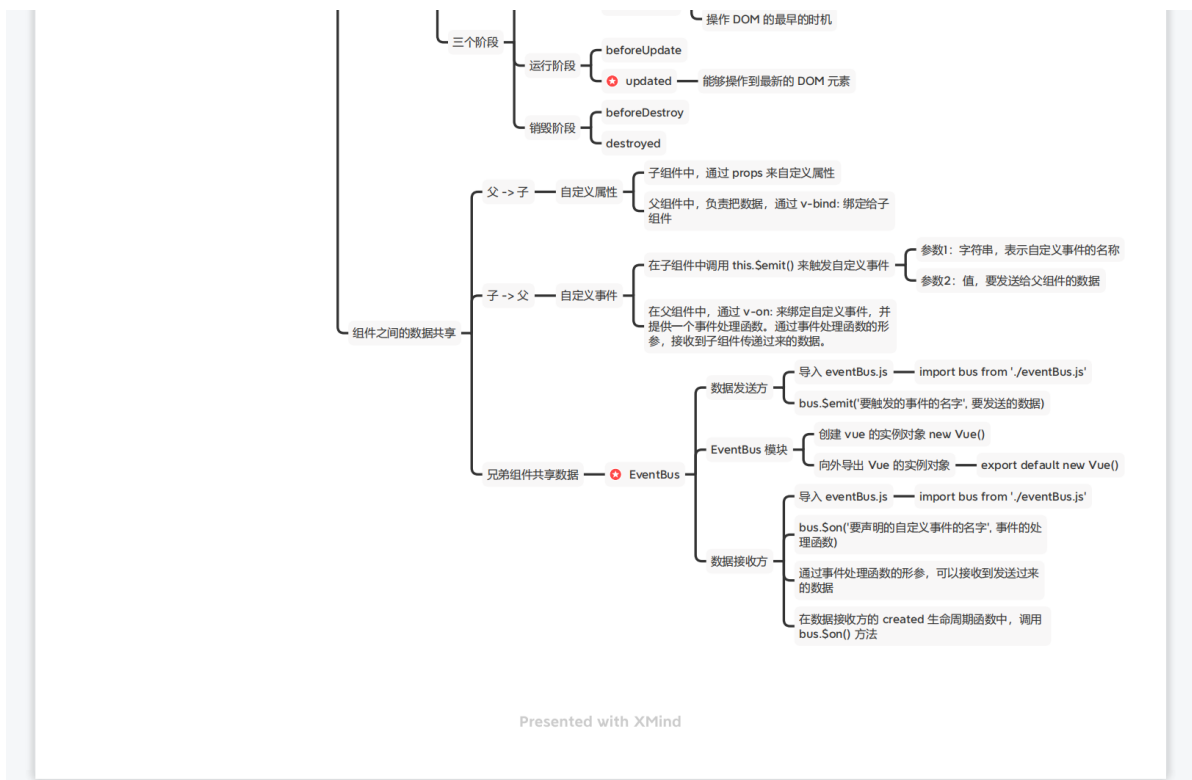


## day2



## day3





## 11.1 Vue2

### 11.1.1 Vue简介

#### 1-1 vue 框架的特性

// 数据驱动视图  
// 双向数据绑定

#### 1-2 数据驱动视图

在使用了 vue 的页面中, vue 会监听数据的变化, 从而自动重新渲染页面的结构。示意图如下:



// 好处: 当页面数据发生变化时, 页面会自动重新渲染!  
// 注意: 数据驱动视图是单向的数据绑定。

#### 1-3 双向数据绑定

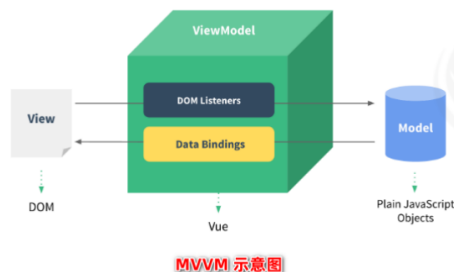
// 在填写表单时, 双向数据绑定可以辅助开发者在不操作 DOM 的前提下, 自动把用户填写的内容同步到数据源中。示意图如下:



好处：开发者不再需要手动操作 DOM 元素，来获取表单元素最新的值！

## 1-4 MVVM

// MVVM 是 vue 实现数据驱动视图和双向数据绑定的核心原理。MVVM 指的是 Model、View 和 ViewModel，它把每个 HTML 页面都拆分成了这三个部分，如图所示：



在 MVVM 概念中！

**Model** 表示当前页面渲染时所依赖的数据源。

**View** 表示当前页面所渲染的 DOM 结构。

**ViewModel** 表示 vue 的实例，它是 MVVM 的核心。

## 1-5 MVVM 的工作原理

ViewModel 作为 MVVM 的核心，是它把当前页面的数据源（Model）和页面的结构（View）连接在了一起



当数据源发生变化时，会被 ViewModel 监听到，VM 会根据最新的数据源自动更新页面的结构

当表单元素的值发生变化时，也会被 VM 监听到，VM 会把变化过后最新的值自动同步到 Model 数据源中

## 1-6 框架

官方给 vue 的定位是**前端框架**，因为它**提供了构建用户界面的一整套解决方案**（俗称 vue 全家桶）：

- vue（核心库）
- vue-router（路由方案）
- vuex（状态管理方案）
- vue 组件库（快速搭建页面 UI 效果的方案）

以及**辅助 vue 项目开发**的一系列工具：

- vue-cli（npm 全局包：一键生成工程化的 vue 项目 - 基于 webpack、大而全）
- vite（npm 全局包：一键生成工程化的 vue 项目 - 小而巧）
- vue-devtools（浏览器插件：辅助调试的工具）
- vetur（vscode 插件：提供语法高亮和智能提示）

## 11.1.2 基础使用

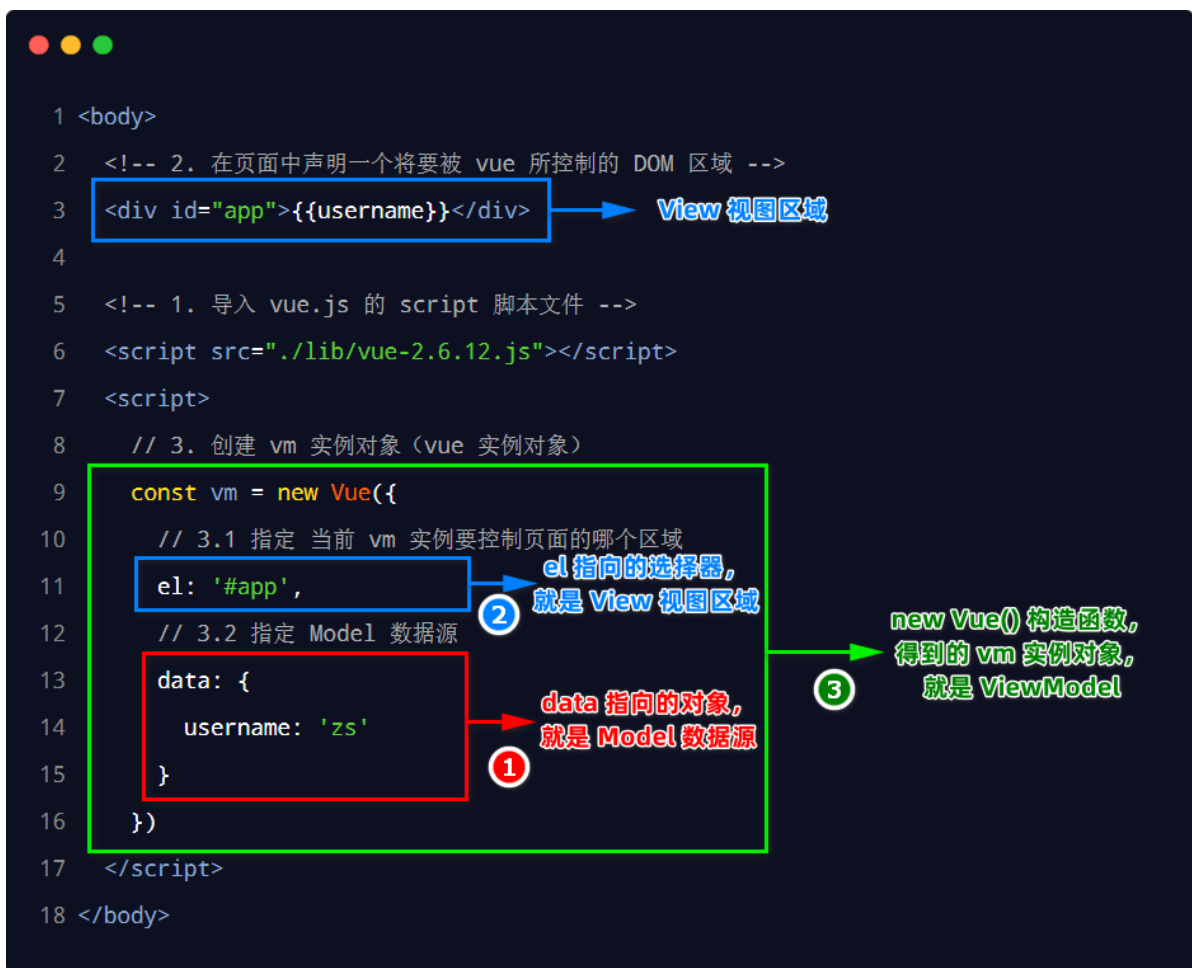
### 2-1 基本使用步骤

```
// 导入 vue.js 的 script 脚本文件
// 在页面中声明一个将要被 vue 所控制的 DOM 区域
// 创建 vm 实例对象（vue 实例对象）
```

```
<body>
 <!-- 2. 在页面中声明一个将要被 vue 所控制的 DOM 区域 -->
 <div id="app">{{username}}</div>

 <!-- 1. 导入 vue.js 的 script 脚本文件 -->
 <script src="./lib/vue-2.6.12.js"></script>
 <script>
 // 3. 创建 vm 实例对象（vue 实例对象）
 const vm = new Vue({
 // 3.1 指定当前 vm 实例要控制页面的哪个区域
 el: '#app',
 // 3.2 指定 Model 数据源
 data: {
 username: 'zs'
 }
 })
 </script>
</body>
```

### 2-2 基本代码与 MVVM 的对应关系



## 2-3 Vue的调试工具

### 1. 安装 vue-devtools 调试工具

// vue 官方提供的 vue-devtools 调试工具, 能够方便开发者对 vue 项目进行调试与开发。

// Chrome 浏览器在线安装 vue-devtools :

<https://chrome.google.com/webstore/detail/vuejs-devtools/nhdogjmejiglipccpnnanhbldajbpd>

// Firefox 浏览器在线安装 vue-devtools :

<https://addons.mozilla.org/zh-CN/firefox/addon/vue-js-devtools/>

### 2.配置 Chrome 浏览器中的 vue-devtools

点击 Chrome 浏览器右上角的 按钮, 选择更多工具 -> 扩展程序 -> Vue.js devtools 详细信息, 并勾选如下的两个选项:

权限

- 读取和更改您在访问的网站上的所有数据

有权访问的网站

允许此扩展程序读取和更改您在所访问的网站上留存的所有数据：

☐ 点击时

☐ 在特定网站上

☒ 在所有网站上

在无痕模式下启用

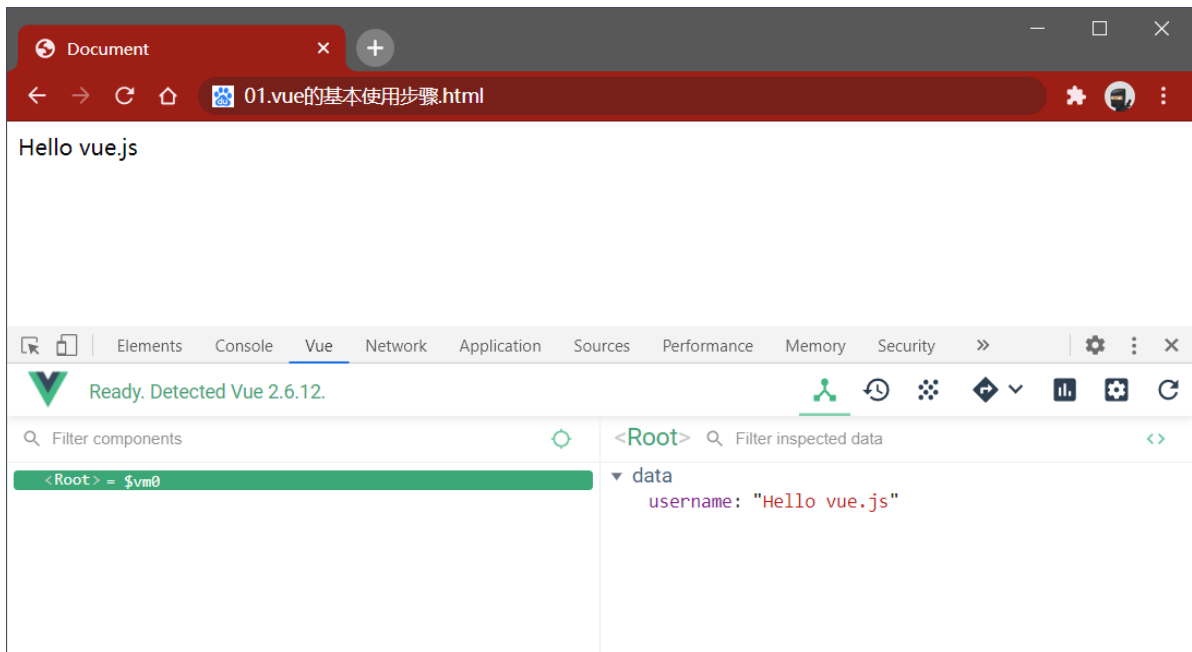
警告：Google Chrome 无法阻止扩展程序记录您的浏览活动。要在无痕模式下停用该扩展程序，请取消选中此选项。

允许访问文件网址

在 Chrome 网上应用店中查看详情

### 3.使用 vue-devtools 调试 vue 页面

在浏览器中访问一个使用了 vue 的页面，打开浏览器的开发者工具，切换到 Vue 面板，即可使用 vue-devtools 调试当前的页面。



## 11.1.3 Vue的指令与过滤器

### 3-1 指令的概念

```
// 指令（Directives）是 vue 为开发者提供的模板语法，用于辅助开发者渲染页面的基本结构。
// vue 中的指令按照不同的用途可以分为如下 6 大类：
// ① 内容渲染指令
// ② 属性绑定指令
// ③ 事件绑定指令
// ④ 双向绑定指令
// ⑤ 条件渲染指令
// ⑥ 列表渲染指令
// 注意：指令是 vue 开发中最基础、最常用、最简单的知识点
```

## 3-2 内容渲染指令

```
<!-- 内容渲染指令用来辅助开发者渲染 DOM 元素的文本内容。常用的内容渲染指令有如下 3 个：-->
v-text
{{ }}
v-html
```

### 2-1 v-text

```
1 <!-- 把 username 对应的值，渲染到第一个 p 标签中 -->
2 <p v-text="username"></p>
3
4 <!-- 把 gender 对应的值，渲染到第二个 p 标签中 -->
5 <!-- 注意：第二个 p 标签中，默认的文本“性别”会被 gender 的值覆盖掉 -->
6 <p v-text="gender">性别</p>
```

注意：v-text 指令会覆盖元素内默认的值。

### 2-2 {{ }} 语法

// vue 提供的 {{ }} 语法，专门用来解决 v-text 会覆盖默认文本内容的问题。这种 {{ }} 语法的专业名称是插值表达式（英文名为：Mustache）。

```
1 <!-- 使用 {{ }} 插值表达式，将对应的值渲染到元素的内容节点中， -->
2 <!-- 同时保留元素自身的默认值 -->
3 <p>姓名：{{username}}</p>
4 <p>性别：{{gender}}</p>
```

注意：相对于 v-text 指令来说，插值表达式在开发中更常用一些！因为它不会覆盖元素中默认的文本内容。

### 2-3 v-html

// v-text 指令和插值表达式只能渲染纯文本内容。如果要把包含 HTML 标签的字符串渲染为页面的 HTML 元素，则需要用到 v-html 这个指令：

```
1 <!-- 假设 data 中定义了名为 discription 的数据，数据的值为包含 HTML 标签的字符串： -->
2 <!-- '<h5 style="color: red;">我在黑马程序员学习 vue.js 课程。</h5>' -->
3
4 <p v-html="discription"></p>
```

### 3-3 属性绑定指令

如果需要为元素的属性动态绑定属性值，则需要用到 v-bind 属性绑定指令。用法示例如下：

```
1 <!-- 假设有如下的 data 数据：
2 data: {
3 inputValue: '请输入内容',
4 imgSrc: 'https://cn.vuejs.org/images/logo.png'
5 }
6 -->
7
8 <!-- 使用 v-bind 指令，为 input 的 placeholder 动态绑定属性值 -->
9 <input type="text" v-bind:placeholder="inputValue" />
10

11 <!-- 使用 v-bind 指令，为 img 的 src 动态绑定属性值 -->
12
```

### 3-1 属性绑定指令的简写形式

由于 v-bind 指令在开发中使用频率非常高，因此，vue 官方为其提供了简写形式（简写为英文的:）。

```
1 <!-- 假设有如下的 data 数据：
2 data: {
3 inputValue: '请输入内容',
4 imgSrc: 'https://cn.vuejs.org/images/logo.png'
5 }
6 -->
7
8 <!-- 使用 v-bind 指令，为 input 的 placeholder 动态绑定属性值 -->
9 <input type="text" :placeholder="inputValue" />
10

11 <!-- 使用 v-bind 指令，为 img 的 src 动态绑定属性值 -->
12
```



### 3-2 使用 Javascript 表达式

在 vue 提供的模板渲染语法中，除了支持绑定简单的数据值之外，还支持 Javascript 表达式的运算，例如：

```
1 {{ number + 1 }}
2
3 {{ ok ? 'YES' : 'NO' }}
4
5 {{ message.split('').reverse().join('') }}
6
7 <div v-bind:id="'list-' + id"></div>
```

### 3-4 事件绑定指令

vue 提供了 v-on 事件绑定指令，用来辅助程序员为 DOM 元素绑定事件监听。语法格式如下：

```
1 <h3>count 的值为: {{count}}</h3>
2 <!-- 语法格式为 v-on:事件名称="事件处理函数的名称" -->
3 <button v-on:click="addCount">+1</button>
```

注意：原生 DOM 对象有 onclick、oninput、onkeyup 等原生事件，替换为 vue 的事件绑定形式后，分别为：v-on:click、v-on:input、v-on:keyup

通过 v-on 绑定的事件处理函数，需要在 methods 节点中进行声明：

```
1 const vm = new Vue({
2 el: '#app',
3 data: { count: 0 },
4 methods: { // v-on 绑定的事件处理函数，需要声明在 methods 节点中
5 addCount() { // 事件处理函数的名字
6 // this 表示当前 new 出来的 vm 实例对象，
7 // 通过 this 可以访问到 data 中的数据
8 this.count += 1
9 }
10 },
11 })
```

## 4-1 事件绑定的简写形式

由于 v-on 指令在开发中使用频率非常高，因此，vue 官方为其提供了简写形式（简写为英文的 @）。

```
1 <div id="app">
2 <h3>count 的值为: {{count}}</h3>
3
4 <!-- 完整写法 -->
5 <button v-on:click="addCount">+1</button>
6
7 <!-- 简写形式，把 v-on: 简写为 @ 符号 -->
8 <!-- 如果事件处理函数中的代码足够简单，只有一行代码，则可以简写到行内 -->
9 <button @click="count += 1">+1</button>
10 </div>
```

## 4-2 事件参数对象

在原生的 DOM 事件绑定中，可以在事件处理函数的形参处，接收事件参数对象 event。同理，在 v-on 指令

（简写为 @）所绑定的事件处理函数中，同样可以接收到事件参数对象 event，示例代码如下

```
1 <h3>count 的值为: {{count}}</h3>
2 <button v-on:click="addCount">+1</button>
3 // -----分割线-----
4 methods: {
5 addCount(e) { // 接收事件参数对象 event，简写为 e
6 const nowBgColor = e.target.style.backgroundColor
7 e.target.style.backgroundColor = nowBgColor === 'red' ? '' : 'red'
8 this.count += 1
9 }
10 }
```

## 4-3 绑定事件并传参

在使用 v-on 指令绑定事件时，可以使用（）进行传参，示例代码如下：

```

1 <h3>count 的值为: {{count}}</h3>
2 <button @click="addNewCount(2)">+2</button>
3 // -----分割线-----
4 methods: {
5 // 在形参处用 step 接收传递过来的参数值
6 addNewCount(step) {
7 this.count += step
8 }
9 }

```

#### 4-4 \$event

\$event 是 vue 提供的特殊变量，用来表示原生的事件参数对象 event。\$event 可以解决事件参数对象 event 被覆盖的问题。示例用法如下：

```

1 <h3>count 的值为: {{count}}</h3>
2 <button @click="addNewCount(2, $event)">+2</button>
3 // -----分割线-----
4 methods: {
5 // 在形参处用 e 接收传递过来的原生事件参数对象 $event
6 addNewCount(step, e) {
7 const nowBgColor = e.target.style.backgroundColor
8 e.target.style.backgroundColor = nowBgColor === 'cyan' ? '' : 'cyan'
9 this.count += step
10 }
11 }

```

#### 4-5 事件修饰符

在事件处理函数中调用 event.preventDefault() 或 event.stopPropagation() 是非常常见的需求。因此，

vue 提供了事件修饰符的概念，来辅助程序员更方便的对事件的触发进行控制。常用的 5 个事件修饰符如下：

事件修饰符	说明
.prevent	阻止默认行为（例如：阻止 a 连接的跳转、阻止表单的提交等）
.stop	阻止事件冒泡
.capture	以捕获模式触发当前的事件处理函数
.once	绑定的事件只触发1次
.self	只有在 event.target 是当前元素自身时触发事件处理函数

```
1 <!-- 触发 click 点击事件时，阻止 a 链接的默认跳转行为 -->
2 百度首页
```

#### 4-6 按键修饰符

在监听键盘事件时，我们经常需要判断详细的按键。此时，可以为键盘相关的事件添加按键修饰符，例如

```
1 <!-- 只有在 `key` 是 `Enter` 时调用 `vm.submit()` -->
2 <input @keyup.enter="submit">
3
4 <!-- 只有在 `key` 是 `Esc` 时调用 `vm.clearInput()` -->
5 <input @keyup.esc="clearInput">
```

#### 3-5 双向绑定指令

vue 提供了 v-model 双向数据绑定指令，用来辅助开发者在不操作 DOM 的前提下，快速获取表单的数据。

```
1 <p>用户名是: {{username}}</p>
2 <input type="text" v-model="username" />
3
4 <p>选中的省份是: {{province}}</p>
5 <select v-model="province">
6 <option value="">请选择</option>
7 <option value="1">北京</option>
8 <option value="2">河北</option>
9 <option value="3">黑龙江</option>
10 </select>
```

#### 5-1 v-model 指令的修饰符

为了方便对用户输入的内容进行处理，vue 为 v-model 指令提供了 3 个修饰符，分别是：

修饰符	作用	示例
.number	自动将用户的输入值转为数值类型	<input v-model.number="age" />
.trim	自动过滤用户输入的首尾空白字符	<input v-model.trim="msg" />
.lazy	在 “change” 时而非 “input” 时更新	<input v-model.lazy="msg" />

```
1 <input type="text" v-model.number="n1"> +
2 <input type="text" v-model.number="n2"> =
3 {{n1 + n2}}
```

### 3-6 条件渲染指令

条件渲染指令用来辅助开发者按需控制 DOM 的显示与隐藏。条件渲染指令有如下两个，分别是：

v-if

v-show

```
1 <div id="app">
2 <p v-if="networkState === 200">请求成功 --- 被 v-if 控制</p>
3 <p v-show="networkState === 200">请求成功 --- 被 v-show 控制</p>
4 </div>
```

#### 6-1 v-if 和 v-show 的区别

```
// 实现原理不同：
// v-if 指令会动态地创建或移除 DOM 元素，从而控制元素在页面上的显示与隐藏；
// v-show 指令会动态为元素添加或移除 style="display: none;" 样式，从而控制元素的显示与隐藏；

// 性能消耗不同：
// v-if 有更高的切换开销，而 v-show 有更高的初始渲染开销。因此：
// 如果需要非常频繁地切换，则使用 v-show 较好
// 如果在运行时条件很少改变，则使用 v-if 较好
```

#### 6-2 v-else

v-if 可以单独使用，或配合 v-else 指令一起使用：

```
1 <div v-if="Math.random() > 0.5">
2 随机数大于 0.5
3 </div>
4 <div v-else>
5 随机数小于或等于 0.5
6 </div>
```

注意：v-else 指令必须配合 v-if 指令一起使用，否则它将被不会被识别！

### 6-3 v-else-if

v-else-if 指令，顾名思义，充当 v-if 的“else-if 块”，可以连续使用：

```
1 <div v-if="type === 'A'">优秀</div>
2 <div v-else-if="type === 'B'">良好</div>
3 <div v-else-if="type === 'C'">一般</div>
4 <div v-else>差</div>
```

注意：v-else-if 指令必须配合 v-if 指令一起使用，否则它将不会被识别！

## 3-7 列表渲染指令

vue 提供了 v-for 列表渲染指令，用来辅助开发者基于一个数组来循环渲染一个列表结构。v-for 指令需要使用 item in items 形式的特殊语法，其中：

items 是待循环的数组

item 是被循环的每一项

```
1 data: {
2 list: [// 列表数据
3 { id: 1, name: 'zs' },
4 { id: 2, name: 'ls' }
5]
6 }
7 // -----分割线-----
8
9 <li v-for="item in list">姓名是: {{item.name}}
10
```

### 7-1 v-for 中的索引

v-for 指令还支持一个可选的第二个参数，即当前项的索引。语法格式为 (item, index) in items，示例代码如下：

```

1 data: {
2 list: [// 列表数据
3 { id: 1, name: 'zs' },
4 { id: 2, name: 'ls' }
5]
6 }
7 // -----分割线-----
8
9 <li v-for="(item, index) in list">索引是: {{index}}, 姓名是: {{item.name}}
10

```

注意: v-for 指令中的 item 项和 index 索引都是形参, 可以根据需要进行重命名。例如 (user, i) in userList

## 7-2 使用 key 维护列表的状态

// 当列表的数据变化时, 默认情况下, **vue** 会尽可能的复用已存在的 **DOM** 元素, 从而提升渲染的性能。但这种默认的性能优化策略, 会导致有状态的列表无法被正确更新。

// 为了给 **vue** 一个提示, 以便它能跟踪每个节点的身份, 从而在保证有状态的列表被正确更新的前提下, 提升渲染的性能。此时, 需要为每项提供一个唯一的 **key** 属性:

```

1 <!-- 用户列表区域 -->
2
3 <!-- 加 key 属性的好处: -->
4 <!-- 1. 正确维护列表的状态 -->
5 <!-- 2. 复用现有的 DOM 元素, 提升渲染的性能 -->
6 <li v-for="user in userList" :key="user.id">
7 <input type="checkbox" />
8 姓名: {{user.name}}
9
10

```

## 7-3 key 的注意事项

// **key** 的值只能是字符串或数字类型  
 // **key** 的值必须具有唯一性 (即: **key** 的值不能重复)  
 // 建议把数据项 **id** 属性的值作为 **key** 的值 (因为 **id** 属性的值具有唯一性)  
 // 使用 **index** 的值当作 **key** 的值没有任何意义 (因为 **index** 的值不具有唯一性)  
 // 建议使用 **v-for** 指令时一定要指定 **key** 的值 (既提升性能、又防止列表状态紊乱)

## 3-8 过滤器

过滤器 (Filters) 是 vue 为开发者提供的功能，常用于文本的格式化。过滤器可以用在两个地方：插值表达式和 v-bind 属性绑定。

过滤器应该被添加在 JavaScript 表达式的尾部，由“管道符”进行调用，示例代码如下：

```
1 <!-- 在双花括号中通过“管道符”调用 capitalize 过滤器，对 message 的值进行格式化 -->
2 <p>{{ message | capitalize }}</p>
3
4 <!-- 在 v-bind 中通过“管道符”调用 formatId 过滤器，对 rawId 的值进行格式化 -->
5 <div v-bind:id="rawId | formatId"></div>
```

## 8-1 定义过滤器

在创建 vue 实例期间，可以在 filters 节点中定义过滤器，示例代码如下

```
1 const vm = new Vue({
2 el: '#app',
3 data: {
4 message: 'hello vue.js',
5 info: 'title info'
6 },
7 filters: { // 在 filters 节点下定义“过滤器”
8 capitalize(str) { // 把首字母转为大写的过滤器
9 return str.charAt(0).toUpperCase() + str.slice(1)
10 }
11 }
12 })
```

## 8-2 私有过滤器和全局过滤器

在 filters 节点下定义的过滤器，称为“私有过滤器”，因为它只能在当前 vm 实例所控制的 el 区域内使用。

如果希望在多个 vue 实例之间共享过滤器，则可以按照如下的格式定义全局过滤器：



```

1 // 全局过滤器 - 独立于每个 vm 实例之外
2 // Vue.filter() 方法接收两个参数:
3 // 第 1 个参数, 是全局过滤器的"名字"
4 // 第 2 个参数, 是全局过滤器的"处理函数"
5 Vue.filter('capitalize', (str) => {
6 return str.charAt(0).toUpperCase() + str.slice(1) + '~~'
7 })

```

### 8-3 连续调用多个过滤器

过滤器可以串联地进行调用, 例如:

```

1 <!-- 把 message 的值, 交给 filterA 进行处理 -->
2 <!-- 把 filterA 处理的结果, 再交给 filterB 进行处理 -->
3 <!-- 最终把 filterB 处理的结果, 作为最终的值渲染到页面上 -->
4 {{ message | filterA | filterB }}

```

示例代码如下:

```

1 <!-- 串联调用多个过滤器 -->
2 <p>{{text | capitalize | maxLength}}</p>
3
4 // 全局过滤器 - 首字母大写
5 Vue.filter('capitalize', (str) => {
6 return str.charAt(0).toUpperCase() + str.slice(1) + '~~'
7 })
8
9 // 全局过滤器 - 控制文本的最大长度
10 Vue.filter('maxLength', (str) => {
11 if (str.length <= 10) return str
12 return str.slice(0, 11) + '...'
13 })

```

### 8-4 过滤器传参

过滤器的本质是 JavaScript 函数, 因此可以接收参数, 格式如下

```

1 <!-- arg1 和 arg2 是传递给 filterA 的参数 -->
2 <p>{{ message | filterA(arg1, arg2) }}</p>
3
4 // 过滤器处理函数的形参列表中：
5 // 第一个参数：永远都是“管道符”前面待处理的值
6 // 从第二个参数开始，才是调用过滤器时传递过来的 arg1 和 arg2 参数
7 Vue.filter('filterA', (msg, arg1, arg2) => {
8 // 过滤器的代码逻辑...
9 })

```

示例代码如下

```

1 <!-- 调用 maxLength 过滤器时传参 -->
2 <p>{{text | capitalize | maxLength(5)}}</p>
3
4 // 全局过滤器 - 首字母大写
5 Vue.filter('capitalize', (str) => {
6 return str.charAt(0).toUpperCase() + str.slice(1) + '~~'
7 })
8
9 // 全局过滤器 - 控制文本的最大长度
10 Vue.filter('maxLength', (str, len = 10) => {
11 if (str.length <= len) return str
12 return str.slice(0, len) + '...'
13 })

```

## 8-5 过滤器的兼容性

```

// 过滤器仅在 vue 2.x 和 1.x 中受支持，在 vue 3.x 的版本中剔除了过滤器相关的功能。

// 在企业级项目开发中：
// 如果使用的是 2.x 版本的 vue，则依然可以使用过滤器相关的功能
// 如果项目已经升级到了 3.x 版本的 vue，官方建议使用计算属性或方法代替被剔除的过滤器功能
// 具体的迁移指南，请参考 vue 3.x 的官方文档给出的说明：

// https://v3.vuejs.org/guide/migration/filters.html#migration-strategy

```

## 11.1.4 侦听器

## 4-1 什么是侦听器

watch 侦听器允许开发者监视数据的变化，从而针对数据的变化做特定的操作。语法格式如下

```
1 const vm = new Vue({
2 el: '#app',
3 data: { username: '' },
4 watch: {
5 // 监听 username 值的变化
6 // newVal 是“变化后的新值”，oldVal 是“变化之前的旧值”
7 username(newVal, oldVal) {
8 console.log(newVal, oldVal)
9 }
10 }
11 })
```

## 4-2 使用 watch 检测用户名是否可用

```
// 监听 username 值的变化，并使用 axios 发起 Ajax 请求，检测当前输入的用户名是否可用：
watch: {
 // 监听 username 值的变化
 async username(newVal) {
 if (newVal === '') return
 // 使用 axios 发起请求，判断用户名是否可用
 const { data: res } = await axios.get('https://www.escook.cn/api/finduser/'
+ newVal)
 console.log(res)
 }
}
```

## 4-3 immediate 选项

默认情况下，组件在初次加载完毕后不会调用 watch 侦听器。如果想让 watch 侦听器立即被调用，则需要使用 immediate 选项。示例代码如下：

```
watch: {
 username: {
 // handler 是固定写法，表示当 username 的值变化时，自动调用 handler 处理函数
 handler: async function (newVal) {
 if (newVal === '') return
 const { data: res } = await
axios.get('https://www.escook.cn/api/finduser/' + newVal)
 console.log(res)
 },
 // 表示页面初次渲染好之后，就立即触发当前的 watch 侦听器
 immediate: true
 }
}
```

## 4-4 deep选项

如果 watch 侦听的是一个对象，如果对象中的属性值发生了变化，则无法被监听到。此时需要使用 deep 选项，代码示例如下

```
1 const vm = new Vue({
2 el: '#app',
3 data: {
4 info: { username: 'admin' }
5 },
6 watch: {
7 info: {
8 handler(newVal) {
9 console.log(newVal.username)
10 },
11 deep: true
12 }
13 }
14 })
```

## 4-5 监听对象单个属性的变化

如果只想监听对象中单个属性的变化，则可以按照如下的方式定义 watch 侦听器：

```
1 const vm = new Vue({
2 el: '#app',
3 data: {
4 info: { username: 'admin' }
5 },
6 watch: {
7 'info.username': {
8 handler(newVal) {
9 console.log(newVal)
10 }
11 }
12 }
13 })
```

## 4-6 计算属性 vs 侦听器

计算属性和侦听器侧重的应用场景不同：

计算属性侧重于监听多个值的变化，最终计算并返回一个新值

侦听器侧重于监听单个数据的变化，最终执行特定的业务处理，不需要有任何返回值

## 11.1.5 计算属性

### 5-1 什么是计算属性

计算属性指的是通过一系列运算之后，最终得到一个属性值。

这个动态计算出来的属性值可以被模板结构或 methods 方法使用。示例代码如下：

```
1 var vm = new Vue({
2 el: '#app',
3 data: {
4 r: 0, g: 0, b: 0
5 },
6 computed: {
7 rgb() { return `rgb(${this.r}, ${this.g}, ${this.b})` }
8 },
9 methods: {
10 show() { console.log(this.rgb) }
11 },
12 })
```

### 5-2 计算属性的特点

虽然计算属性在声明的时候被定义为方法，但是计算属性的本质是一个属性

计算属性会缓存计算的结果，只有计算属性依赖的数据变化时，才会重新进行运算

### 5-3 如何声明计算属性

```
1 <input type="text" v-model.number="count" />
2 <p>{{ count }} 乘以 2 的值为: {{ plus }}</p>
3
4 data() {
5 return { count: 1 }
6 },
7 computed: {
8 plus() { // 计算属性: 监听 data 中 count 值的变化, 自动计算出 count * 2 之后的新值
9 return this.count * 2
10 },
11 }
```

### 5-4 计算属性的使用注意点

- ① 计算属性必须定义在 computed 节点中
- ② 计算属性必须是一个 function 函数
- ③ 计算属性必须有返回值
- ④ 计算属性必须当做普通属性使用

## 5-5 计算属性 vs 方法

相对于方法来说，计算属性会缓存计算的结果，只有计算属性的依赖项发生变化时，才会重新进行运算。因此计算属性的性能更好

```
1 computed: {
2 plus() { // 计算属性的计算结果会被缓存，性能好
3 console.log('计算属性被执行了')
4 return this.count * 2
5 },
6 },
7 methods: {
8 plus() { // 方法的计算结果无法被缓存，性能低
9 console.log('方法被执行了')
10 return this.count * 2
11 }
12 },
```

## 11.1.6 vue-cli

### 6-1 单页面应用程序

单页面应用程序（英文名：Single Page Application）简称 SPA，顾名思义，指的是一个 Web 网站中只有唯一的一个 HTML 页面，所有的功能与交互都在这唯一的一个页面内完成。

#### 1-1 单页面应用程序的特点

单页面应用程序将所有的功能局限于一个 web 页面中，仅在该 web 页面初始化时加载相应的资源（HTML、JavaScript 和 CSS）。

一旦页面加载完成了，SPA 不会因为用户的操作而进行页面的重新加载或跳转。而是利用 JavaScript 动态地变换 HTML 的内容，从而实现页面与用户的交互。

#### 1-2 单页面应用程序的优点

SPA 单页面应用程序最显著的 3 个优点如下：

##### ① 良好的交互体验

- 单页应用的内容的改变不需要重新加载整个页面
- 获取数据也是通过 Ajax 异步获取
- 没有页面之间的跳转，不会出现“白屏现象”

##### ② 良好的前后端工作分离模式

- 后端专注于提供 API 接口，更易实现 API 接口的复用
- 前端专注于页面的渲染，更利于前端工程化的发展

##### ③ 减轻服务器的压力

- 服务器只提供数据，不负责页面的合成与逻辑的处理，吞吐能力会提高几倍

### 1-3 单页面应用程序的缺点

任何一种技术都有自己的局限性，对于 SPA 单页面应用程序来说，主要的缺点有如下两个：

#### ① 首屏加载慢

- 路由懒加载
- 代码压缩
- CDN 加速
- 网络传输压缩

#### ② 不利于 SEO

- SSR 服务器端渲染

### 1-4 如何快速创建 vue 的 SPA 项目

vue 官方提供了两种快速创建工程化的 SPA 项目的方式：

#### ① 基于 vite 创建 SPA 项目

#### ② 基于 vue-cli 创建 SPA 项目

	vite	vue-cli
支持的 vue 版本	仅支持 vue3.x	支持 3.x 和 2.x
是否基于 webpack	否	是
运行速度	快	较慢
功能完整度	小而巧（逐渐完善）	大而全
是否建议在企业级开发中使用	目前不建议	建议在企业级开发中使用

## 6-2 什么是vue-cli

vue-cli 是 Vue.js 开发的标准工具。它简化了程序员基于 webpack 创建工程化的 Vue 项目的过程。

引用自 vue-cli 官网上的一句话：程序员可以专注在撰写应用上，而不必花好几天去纠结 webpack 配置的问题。

中文官网：<https://cli.vuejs.org/zh/>

## 6-3 安装和使用

```
// vue-cli 是 npm 上的一个全局包，使用 npm install 命令，即可方便的把它安装到自己的电脑上：
// npm install -g @vue/cli
// 基于 vue-cli 快速生成工程化的 Vue 项目：
vue create 项目的名称
```

## 6-4 vue 项目的运行流程

```
在工程化的项目中，vue 要做的事情很简单：通过 main.js 把 App.vue 渲染到 index.html 的指定区域中。
```

```
其中：
```

```
① App.vue 用来编写待渲染的模板结构
```

```
② index.html 中需要预留一个 el 区域
```

```
③ main.js 把 App.vue 渲染到了 index.html 所预留的区域中
```

## 11.1.7 Vue组件

### 7-1 什么是组件化开发

组件化开发指的是：根据封装的思想，把页面上可重用的 UI 结构封装为组件，从而方便项目的开发和维护。

### 7-2 vue中的组件化开发

```
// vue 是一个支持组件化开发的前端框架。
```

```
// vue 中规定：组件的后缀名是 .vue。之前接触到的 App.vue 文件本质上就是一个 vue 的组件
```

### 7-3 vue组件的三个组成部分

```
// 每个 .vue 组件都由 3 部分构成，分别是：
```

```
// template -> 组件的模板结构
```

```
// script -> 组件的 JavaScript 行为
```

```
// style -> 组件的样式
```

```
// 其中，每个组件中必须包含 template 模板结构，而 script 行为和 style 样式是可选的组成部分。
```

#### 3-1 template

vue 规定：每个组件对应的模板结构，需要定义到